

PCCST503 MACHINE LEARNING

**MODULE 2
SEMESTER 5
KTU 2024 SCHEME**

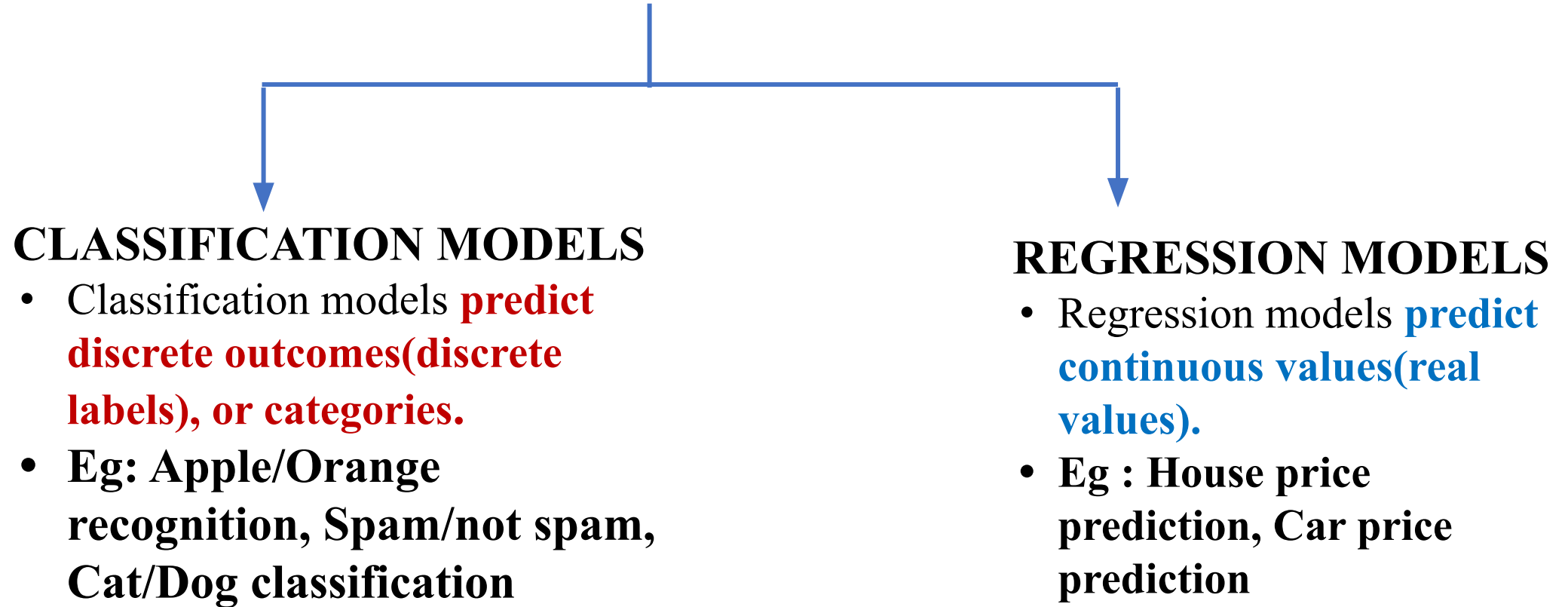
CONTENTS

- **Classification**
 - Logistic regression**

Supervised Learning

- Supervised learning is a machine learning technique where the **model learns from labeled data.**
- In supervised learning, the algorithm is **trained on a labeled dataset.**
- **Labeled data means input(features) is already tagged with correct output(label).**
- Goal for the model is to **learn a mapping function from inputs to outputs.**
- This allows the algorithm to make predictions when given new, unseen data.

TYPES OF SUPERVISED LEARNING MODELS



CLASSIFICATION MODELS

- Classification models **predict discrete outcomes(discrete labels), or categories.**
- **Eg: Apple/Orange recognition, Spam/not spam, Cat/Dog classification**

REGRESSION MODELS

- Regression models **predict continuous values(real values).**
- **Eg : House price prediction, Car price prediction**

Classification

- A typical supervised learning task is classification.
- In classification, the **label represents a categorization, or class.**
- Classification models **predict discrete outcomes(discrete labels), or categories.**
- Goal is to **assign an input data to one of several predefined classes.**
- There are **different types of classification problems** depending on how many **categories** (or classes) we are working with and how they are organized.

i. **Binary Classification**

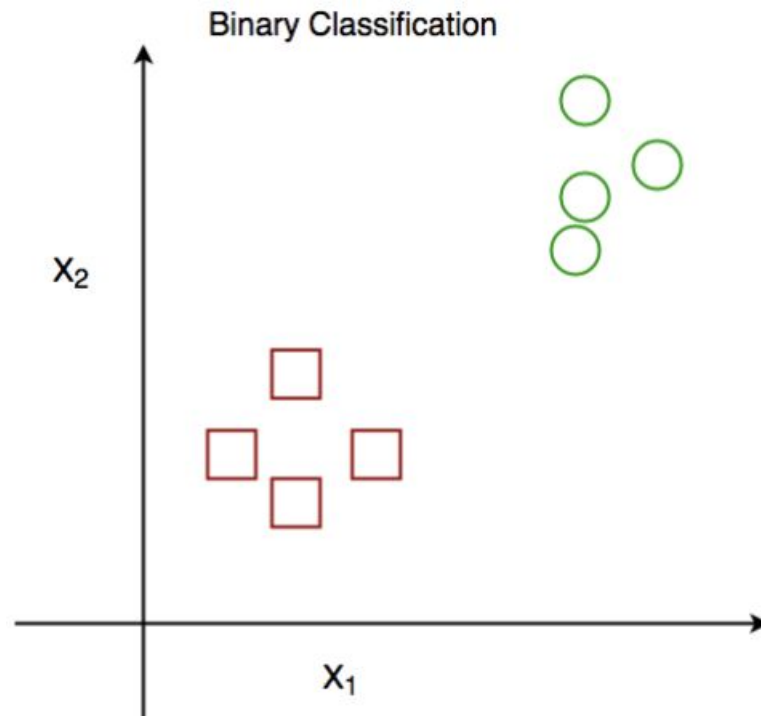
ii. **Multiclass Classification**

iii. **Multilabel Classification**

iv. **Multioutput Classification**

i. Binary classification

- It categorizes data points into one of two distinct classes.
- Logistic Regression or Support Vector Machine classifiers are strictly binary classifiers.
- **Example 1**: classifying emails as spam or non-spam.

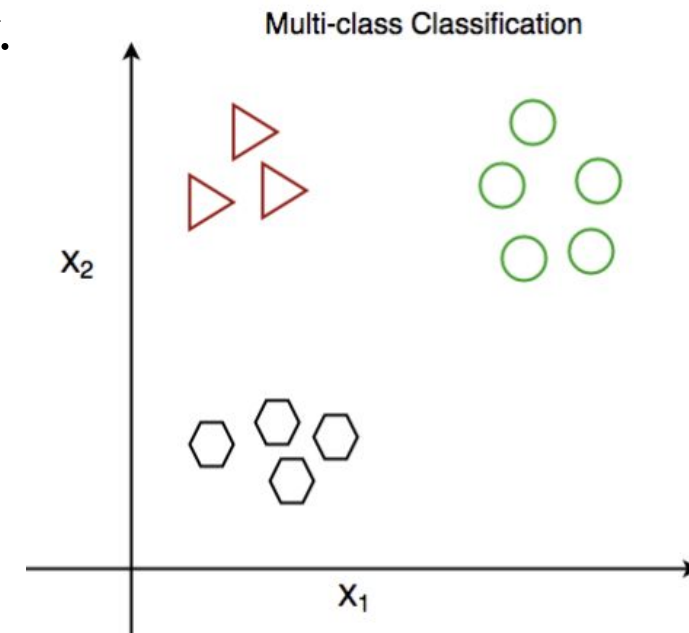


ii. Multiclass classification(Multinomial classification)

- Multiclass classification extends binary classification to **predict a label that represents one of multiple possible classes.**
- **Classifies data into three or more classes.**
- **Multiclass classifiers** (also called **multinomial classifiers**) can distinguish between more than two classes.
- Some algorithms (such as **SGD classifiers, Random Forest classifiers, and naive Bayes classifiers**) are capable of handling multiple classes natively.

Example :

- An example of multiclass classification, using **images of vegetables**, where each image is **either a carrot, tomato, or potato.**



iii. Multilabel Classification

- Some classification tasks require **predicting more than one class label for each input.**
- These tasks are referred to as **Multiple label classification or Multilabel Classification.**
- Consider a **face recognition classifier**:
 - what should it do if it recognizes several people in the same picture?
 - It should attach one tag per person it recognizes.
 - Say the classifier has been trained to recognize three faces, **Alice, Bob, and Charlie.**
 - Then when the classifier is shown a picture of Alice and Charlie, it should output [1, 0, 1] (meaning “Alice yes, Bob no, Charlie yes”).
 - Such a classification system that outputs multiple binary tags is called a multilabel classification system.
- A **movie recommendation system** could **tag a movie as both action and comedy.** The system checks various features (like movie plot, actors, or genre tags) and assigns multiple labels to a single piece of data, rather than just one.

iv. Multioutput Classification(Multioutput–multiclass classification or multitask classification)

- It is simply a generalization of multilabel classification where **each label can be multiclass (i.e., it can have more than two possible values)**.
- For example, classification of the properties **“type of fruit”** and **“colour”** for a set of images of fruit.
- The property **“type of fruit”** has the possible classes: **“apple”, “pear”** and **“orange”**.
- The property **“colour”** has the possible classes: **“green”, “red”, “yellow”** and **“orange”**.
- Each sample is an image of a fruit, a label is output for both properties and **each label is one of the possible classes of the corresponding property**.

Types of Classification models

- **Logistic Regression : Predicts probability of class using a logistic function.**
- **Support Vector Machine(SVM) :** Finds optimal hyperplane separating classes with maximum margin.
- **Naive Bayes classifier :** Uses Bayes' Theorem with strong feature independence.
- **Decision Trees :** Splits data using feature thresholds to form a tree structure.
- **Random Forest Classifiers :** Builds multiple decision trees and combines their output.

Logistic Regression(Logit Regression)

- Despite its name, it's **used for classification, not regression**.
- Logistic Regression is a **supervised machine learning** algorithm **used for classification problems**.
- It is used **for binary classification** where the output can be one of two possible categories such as Yes/No, True/False or 0/1.
- It predicts the probability that an input belongs to a particular class. (e.g., what is the probability that this email is spam?).
- If the **output of the sigmoid function (estimated probability)** is **greater than a predefined threshold(50%) on the graph**, the model predicts that the instance belongs to that class (called the **positive class, labeled “1”**).
- If the estimated probability is less than the predefined threshold, the model predicts that the instance does not belong to the class(i.e., it belongs to **the negative class, labeled “0”**).
- This makes it a binary classifier.

Logistic Regression(Logit Regression)

- It uses a **logistic function** called a **sigmoid function** to map predictions and their probabilities.
- **Sigmoid function** refers to **an S-shaped curve** that convert **inputs into a probability value between 0 and 1**.
- The **sigmoid function** is referred to as an activation function for logistic regression and is defined as:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

Logistic Regression

$\sigma(x)$ tends towards 1 as $x \rightarrow \infty$

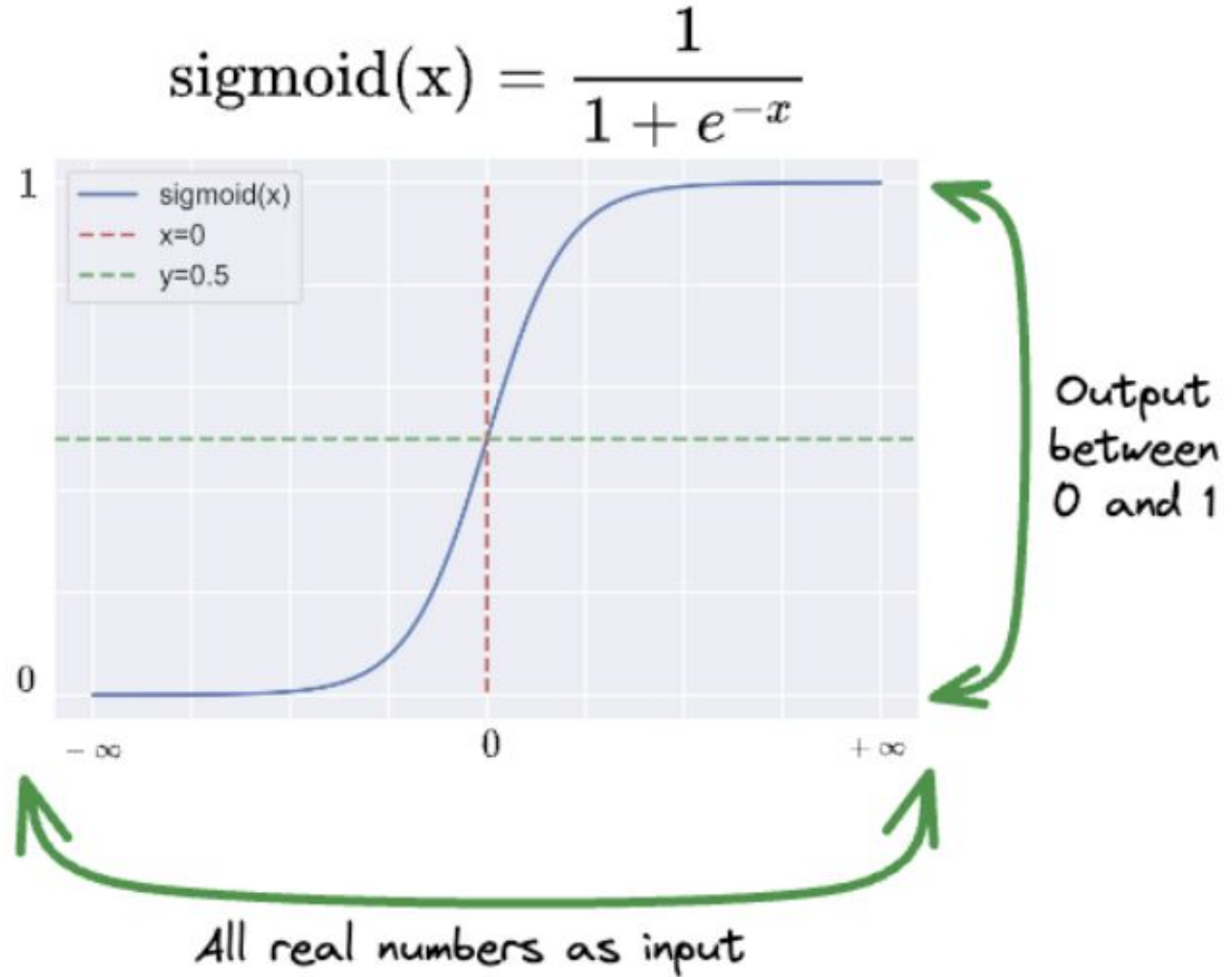
$\sigma(x)$ tends towards 0 as $x \rightarrow -\infty$

$\sigma(x)$ is always bounded between 0 and 1

where the probability of being a class can be measured as:

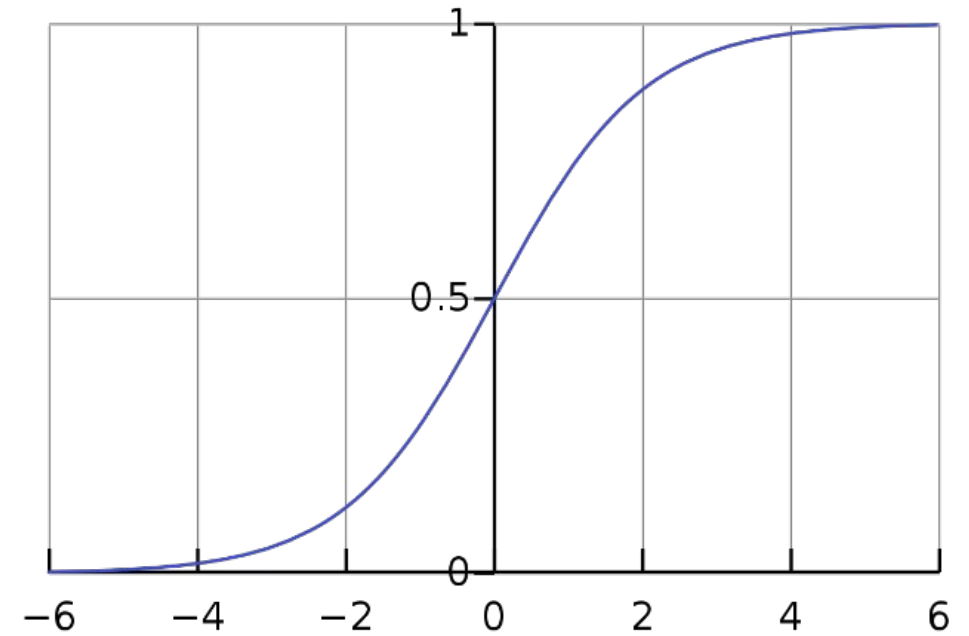
$$P(y = 1) = \sigma(x)$$

$$P(y = 0) = 1 - \sigma(x)$$



Graph of the sigmoid function

- As we can see in Fig 3, we can feed any real number to the sigmoid function and it will return a value between 0 and 1.



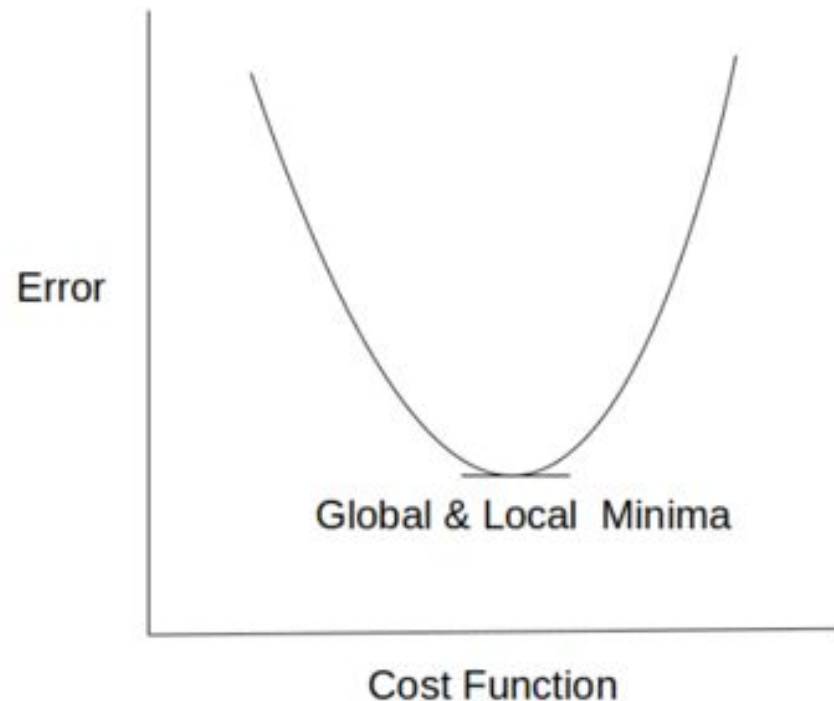
Once the Logistic Regression model has estimated the probability $\hat{p} = h_{\theta}(\mathbf{x})$ that an instance $\mathbf{x}$ belongs to the positive class, it can make its prediction $\hat{y}$ easily (see **Equation 4-15**).

Equation 4-15. Logistic Regression model prediction

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

Cost Function in Logistic Regression - **Log Loss Function**

- **In linear regression**, we use the **Mean squared error** which was the difference between $y_{\text{predicted}}$ and y_{actual} .
- The graph of the cost function in linear regression is like this:



Cost Function in Logistic Regression - **Log Loss Function**

- In logistic regression Y_i is a non-linear function ($\hat{Y} = 1 / (1 + e^{-x})$). If we use this in the above MSE equation then it will give a non-convex graph with many local minima as shown.



The problem here is that this cost function will give results with local minima, which is a big problem because then we'll miss out on our global minima and our error will increase.

In order to solve this problem, we derive a different cost function for logistic regression called **log loss function**.

Cost Function in Logistic Regression - **Log Loss Function**

Why Not Use Mean Squared Error (MSE)

- MSE works well for regression, but **in Logistic Regression it creates a non-convex curve (multiple local minima).**
- Log loss ensures a convex cost function, making optimization with Gradient Descent easier and guaranteeing a global minimum.

Log Loss Function (Cross-entropy loss or Logarithmic loss)

Logistic Regression cost function (Log loss function)

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i) \right]$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \cdot \log(\hat{p}_i) + (1-y_i) \cdot \log(1-\hat{p}_i) \right]$$

where

y_i - actual output of i^{th} sample

$\hat{p}_i$ or $\hat{y}_i$ - probability predicted for i^{th} sample.

Types of Logistic Regression

- Generally, logistic regression means binary logistic regression having binary target variables, but there can be two more categories of target variables that can be predicted by it.
- Based on those number of categories, Logistic regression can be divided into following types.

- 1. Binary or Binomial Logistic regression**
- 2. Multinomial Logistic regression**
- 3. Ordinal Logistic regression**

Binomial Logistic regression

- Classifies data into two classes (e.g., Yes/No, 0/1, Spam/Not Spam)
- The sigmoid function is used for prediction.

Multinomial Logistic Regression(Softmax Regression)

- Target variable can have 3 or more possible types which are not ordered i.e types have no quantitative significance like “disease A” vs “disease B” vs “disease C”.
- In this case, the softmax function is used in place of the sigmoid function.
- Softmax function for N classes will be:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

Here

N represents the number of elements in the vector $\mathbf{z}$ and i, j iterates over all the elements in the vector.

Multinomial Logistic Regression(Softmax Regression)



Example Use Cases:

- Handwritten digit recognition (0–9) 🖋️
- Classifying types of flowers 🌻



Softmax Function Formula:

$$P(y = k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Where:

- z_k = score for class k
- K = number of total classes

Ordinal Logistic Regression

- In such a kind of classification, dependent variable can have 3 or more possible ordered types or the types having a quantitative significance.
- For example, these variables may represent “poor” or “good”, “very good”, “Excellent” and each category can have the scores like 0,1,2,3.

Logistic Regression

- Logistic regression calculates this probability using the sigmoid function applied to the linear combination of the input features which is specified as:

$$P(Y = 1) = \frac{1}{1 + e^{-(b_0 + b_1 X_1 + b_2 X_2 + \dots + b_n X_n)}}$$

Where,

- $P(Y=1)$ = Probability of the data point belonging to the positive class
- $X_1, \dots, X_n$ = Input Features
- $b_0, \dots, b_n$ = Input weights that the algorithm learns during training
- This sigmoid function is in the form of S like curve that transforms any data point to a probability score within the range of 0-1.

Why Logistic Regression is Better for Classification

- Logistic Regression is specifically designed for classification tasks.
- It solves the issues that Linear Regression faces when used for classification.

1. Logistic Regression Outputs Probabilities

- Instead of predicting continuous values, Logistic Regression predicts probabilities.
- It uses a function called the sigmoid function, which takes any input and converts it into a probability between 0 and 1.
- This makes Logistic Regression ideal for binary classification.

Why Logistic Regression is Better for Classification

2. Decision Boundary for Classification

- Logistic Regression establishes a clear decision boundary.
- The decision boundary separates the classes based on the probabilities calculated by the sigmoid function.
- In binary classification, any value greater than 0.5 is classified as “1” (positive class), and anything less than 0.5 is classified as “0” (negative class).

3. Resistant to Outliers

- Logistic Regression is less affected by outliers than Linear Regression.
- While it still considers all data points, its focus is on correctly classifying points based on their distance from the decision boundary.
- This makes it a better fit for classification tasks where data may contain outliers or noise.

Sl No	Linear Regression	Logistic Regression
1	Linear Regression is a supervised regression model.	Logistic Regression is a supervised classification model.
2	Linear regression is used to predict the continuous dependent variable using a given set of independent variables.	Logistic regression is used to predict the categorical dependent variable using a given set of independent variables.
3	It is used for solving regression problem.	It is used for solving classification problems.
4	In this we predict the value of continuous variables	In this we predict values of categorical variables
5	In this we find best fit line.	In this we find S-Curve.

SI No	Linear Regression	Logistic Regression
6	The output must be continuous value, such as price, age etc.	Output must be categorical value such as 0 or 1, Yes or no etc.
7	It required linear relationship between dependent and independent variables.	It not required linear relationship.
8	Here no activation function is used.	Here activation function is used to convert a linear regression equation to the logistic regression equation

SI No	Linear Regression	Logistic Regression
9	Here no threshold value is needed.	Here a threshold value is added.
10	Applications of linear regression: • Financial risk assessment • Business insights • Market analysis	Applications of logistic regression: • Medicine • Credit scoring • Hotel Booking • Gaming • Text editing



BAYES THEOREM AND BAYES CLASSIFIER

NAÏVE BAYES CLASSIFIER

Conditional Probability

- The probability of the occurrence of an event A given that an event B has already occurred is called the conditional probability of A given B and is denoted by $P(A|B)$.

The formula is $P(A|B) = \frac{P(A \cap B)}{P(B)}$, where $P(B) \neq 0$.

- $P(A|B)$ is the probability of event A given that event B occurred.
- $P(A \cap B)$ is the probability of both A and B occurring.
- $P(B)$ is the probability of event B occurring.

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$$P(B|A) = \frac{P(B \cap A)}{P(A)}$$



Independent and Dependent events

- Two events are independent if knowledge that one occurred does not change the probability that other occurred informally Events are independent if they do not influence one another
- Toss a coin twice .The outcome of two tosses are independent of one another.
- Dependent event is affected by outcome of second event.

Bayes theorem

Bayes' Theorem states:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

- $P(B|A)$ is the probability of event B given that event A occurred.
- $P(A)$ is the "prior probability" or the initial probability of A before observing B.
- $P(B)$ is the "evidence" or the probability of event B occurring.
- $P(A|B)$ is the "posterior probability," which is the updated probability of A after considering the evidence B. 

$$\begin{array}{c} \text{Posterior} \\ \downarrow \\ P(A|B) \end{array} = \frac{\begin{array}{c} \text{Likelihood} \\ \downarrow \\ P(B|A) \end{array} * \begin{array}{c} \text{Prior} \\ \downarrow \\ P(A) \end{array}}{\begin{array}{c} \uparrow \\ P(B) \\ \text{Evidence} \end{array}}$$

Bayes theorem- multiclass

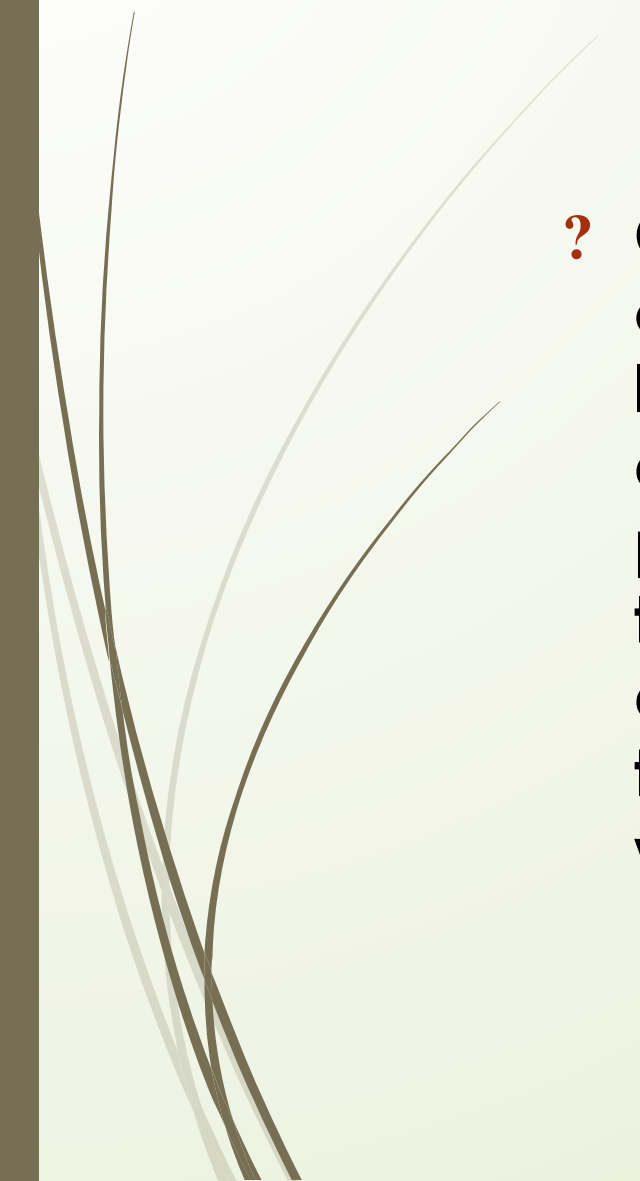
Let the sample space be divided into disjoint events $B_1, B_2, \dots, B_n$ and A be any event. Then we have

$$P(B_k|A) = \frac{P(A|B_k)P(B_k)}{\sum_{i=1}^n P(A|B_i)P(B_i)}$$

- $P(B_k|A)$: This is the posterior probability of event B_k occurring given that event A has occurred. It is the probability we are trying to find.
- $P(A|B_k)$: This is the likelihood, or the probability of event A occurring given that event B_k has occurred.
- $P(B_k)$: This is the prior probability of event B_k . It is the initial probability of the event before any new evidence is considered.
- $\sum_{i=1}^n P(A|B_i)P(B_i)$: This is the total probability of event A occurring. It is the sum of the probabilities of A occurring with each of the disjoint events $B_1, B_2, \dots, B_n$. This part of the formula is often referred to as the law of total probability. It serves as a normalizing constant to ensure the posterior probabilities sum to 1.



Bayes Theorem – Problem 1

- 
- ? Consider a set of patients coming for treatment in a certain clinic. Let A denote the event that a “Patient has liver disease” and B the event that a “Patient is an alcoholic.” It is known from experience that 10% of the patients entering the clinic have liver disease and 5% of the patients are alcoholics. Also, among those patients diagnosed with liver disease, 7% are alcoholics. Given that a patient is alcoholic, what is the probability that he will have liver disease?**

- Let B be the event that a "Patient is an alcoholic."

Based on the problem description, we have the following probabilities:

- $P(A) = 10\% = 0.10$ (Probability a patient has liver disease).
- $P(B) = 5\% = 0.05$ (Probability a patient is an alcoholic).
- $P(B|A) = 7\% = 0.07$ (Probability a patient is an alcoholic, given that they have liver disease).

The goal is to find the probability that a patient will have liver disease given that they are alcoholic, which is $P(A|B)$.

Solution using Bayes' Theorem

Bayes' Theorem states:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Substitute the given values into the formula:

$$P(A|B) = \frac{0.07 \cdot 0.10}{0.05}$$

$$P(A|B) = \frac{0.007}{0.05}$$

$$P(A|B) = 0.14$$

This means that given a patient is alcoholic, there is a 0.14 or 14% probability that they have liver disease.



Bayes Theorem – Problem 2

- ? Three factories A, B, C of an electric bulb manufacturing company produce respectively 35%, 35% and 30% of the total output. Approximately 1.5%, 1% and 2% of the bulbs produced by these factories are known to be defective. If a randomly selected bulb manufactured by the company was found to be defective, what is the probability that the bulb was manufactured in factory A?**

Step 1: Define the probabilities of production for each factory

- $P(A) = 0.35$ (Factory A produces 35% of bulbs)
- $P(B) = 0.35$ (Factory B produces 35% of bulbs)
- $P(C) = 0.30$ (Factory C produces 30% of bulbs) 

Step 2: Define the probabilities of a bulb being defective from each factory

- $P(\text{Defective}|A) = 0.015$ (1.5% of bulbs from A are defective)
- $P(\text{Defective}|B) = 0.01$ (1% of bulbs from B are defective)
- $P(\text{Defective}|C) = 0.02$ (2% of bulbs from C are defective) 

Step 3: Calculate the total probability of a bulb being defective using the law of total probability

$$P(\text{Defective}) = P(\text{Defective}|A)P(A) + P(\text{Defective}|B)P(B) + P(\text{Defective}|C)P(C)$$

$$P(\text{Defective}) = (0.015 \times 0.35) + (0.01 \times 0.35) + (0.02 \times 0.30)$$

$$P(\text{Defective}) = 0.00525 + 0.0035 + 0.006$$

$$P(\text{Defective}) = 0.01475$$

Step 4: Apply Bayes' Theorem to find the probability that the bulb is from factory A, given it is defective

$$P(A|\text{Defective}) = \frac{P(\text{Defective}|A) P(A)}{P(\text{Defective})}$$

$$P(A|\text{Defective}) = \frac{(0.015 \times 0.35)}{0.01475}$$

$$P(A|\text{Defective}) = \frac{0.00525}{0.01475}$$

$$P(A|\text{Defective}) \approx 0.3559$$



Naïve Bayes Classifier

- The naive Bayes algorithm is based on the following assumptions:
 - All the features are independent and are unrelated to each other. Presence or absence of a feature does not influence the presence or absence of any other feature.
 - The data has class-conditional independence, which means that events are independent so long as they are conditioned on the same class value. These assumptions are, in general, true in many real world problems

Basic idea

- Suppose we have a training data set consisting of N examples having n features
- Let the features be named as $(F_1, \dots, F_n)$.
- A feature vector is of the form $(f_1, f_2, \dots, f_n)$.
- Let the set of class labels be $\{c_1, c_2, \dots, c_p\}$.
- Suppose we are given a test instance having the feature vector, $X = (x_1, x_2, \dots, x_n)$.
- We are required to determine the most appropriate class label that should be assigned to the test instance.
- For this purpose we compute the following conditional probabilities :
 $P(c_1|X), P(c_2|X), \dots, P(c_p|X)$.
- choose the maximum among them.
- Suppose the maximum probability be $P(c_i | X)$. Then, we choose c_i as the most appropriate class label for the training instance having X as the feature vector.

Computation of probabilities

Using Bayes' theorem, we have:

$$P(c_k|X) = \frac{P(X|c_k)P(c_k)}{P(X)} \quad (6.6)$$

Since, by assumption, the data has class-conditional independence, we note that the events “ $x_1|c_k$ ”, “ $x_2|c_k$ ”, ..., $x_n|c_k$ are independent (because they are all conditioned on the same class label c_k). Hence we have

$$\begin{aligned} P(X|c_k) &= P((x_1, x_2, \dots, x_n)|c_k) \\ &= P(x_1|c_k)P(x_2|c_k)\cdots P(x_n|c_k) \end{aligned}$$

Using this in Eq.(6.6) we get

$$P(c_k|X) = \frac{P(x_1|c_k)P(x_2|c_k)\cdots P(x_n|c_k)P(c_k)}{P(X)}.$$

Since the denominator $P(X)$ is independent of the class labels, we have

$$P(c_k|X) \propto P(x_1|c_k)P(x_2|c_k)\cdots P(x_n|c_k)P(c_k).$$

So it is enough to find the maximum among the following values:

$$P(x_1|c_k)P(x_2|c_k)\cdots P(x_n|c_k)P(c_k), \quad k = 1, \dots, p.$$

Computation of probabilities

The various probabilities in the above expression are computed as follows:

$$P(c_k) = \frac{\text{No. of examples with class label } c_k}{\text{Total number of examples}}$$

$$P(x_j|c_k) = \frac{\text{No. of examples with } j\text{th feature equal to } x_j \text{ and class label } c_k}{\text{No. of examples with class label } c_k}$$

Naïve Bayes algorithm

Let there be a training data set having n features $F_1, \dots, F_n$. Let f_1 denote an arbitrary value of F_1 , f_2 of F_2 , and so on. Let the set of class labels be $\{c_1, c_2, \dots, c_p\}$. Let there be given a test instance having the feature vector

$$X = (x_1, x_2, \dots, x_n).$$

We are required to determine the most appropriate class label that should be assigned to the test instance.

Step 1. Compute the probabilities $P(c_k)$ for $k = 1, \dots, p$.

Step 2. Form a table showing the conditional probabilities

$$P(f_1|c_k), \quad P(f_2|c_k), \quad \dots, \quad P(f_n|c_k)$$

for all values of $f_1, f_2, \dots, f_n$ and for $k = 1, \dots, p$.

Step 3. Compute the products

$$q_k = P(x_1|c_k)P(x_2|c_k)\cdots P(x_n|c_k)P(c_k)$$

for $k = 1, \dots, p$.

Step 4. Find j such $q_j = \max\{q_1, q_2, \dots, q_p\}$.

Step 5. Assign the class label c_j to the test instance X .

Problem 1

Sl. No.	Swim	Fly	Crawl	Class
1	Fast	No	No	Fish
2	Fast	No	Yes	Animal
3	Slow	No	No	Animal
4	Fast	No	No	Animal
5	No	Short	No	Bird
6	No	Short	No	Bird
7	No	Rarely	No	Animal
8	Slow	No	Yes	Animal
9	Slow	No	No	Fish
10	Slow	No	Yes	Fish
11	No	Long	No	Bird
12	Fast	No	No	Bird

Use naive Bayes algorithm to classify a particular species if its features are (Slow, Rarely, No)?

Solution

In this example, the features are

$$F_1 = \text{"Swim"}, \quad F_2 = \text{"Fly"}, \quad F_3 = \text{"Crawl"}.$$

The class labels are

$$c_1 = \text{"Animal"}, \quad c_2 = \text{"Bird"}, \quad c_3 = \text{"Fish"}.$$

The test instance is (Slow, Rarely, No) and so we have:

$$x_1 = \text{"Slow"}, \quad x_2 = \text{"Rarely"}, \quad x_3 = \text{"No"}.$$

We construct the frequency table shown in Table 6.2 which summarises the data. (It may be noted that the construction of the frequency table is not part of the algorithm.)

Class	Features									Total
	Swim (F_1)			Fly (F_2)				Crawl (F_3)		
	Fast	Slow	No	Long	Short	Rarely	No	Yes	No	
Animal (c_1)	2	2	1	0	0	1	4	2	3	5
Bird (c_2)	1	0	3	1	2	0	1	1	3	4
Fish (c_3)	1	2	0	0	0	0	3	0	3	3
Total	4	4	4	1	2	1	8	4	8	12

Table 6.2: Frequency table for the data in Table 6.1



Step 1. We compute following probabilities.

$$P(c_1) = \frac{\text{No. of records with class label "Animal"}}{\text{Total number of examples}} \\ = 5/12$$

$$P(c_2) = \frac{\text{No. of records with class label "Bird"}}{\text{Total number of examples}} \\ = 4/12$$

$$P(c_3) = \frac{\text{No of records with class label "Fish"}}{\text{Total number of examples}} \\ = 3/12$$

Step 2. We construct the following table of conditional probabilities:

Class	Features								
	Swim (F_1)			Fly (F_2)				Crawl (F_3)	
	f_1			f_2				f_3	
	Fast	Slow	No	Long	Short	Rarely	No	Yes	No
Animal (c_1)	2/5	2/5	1/5	0/5	0/5	1/5	4/5	2/5	3/5
Bird (c_2)	1/4	0/4	3/4	1/4	2/4	0/4	1/4	0/4	4/4
Fish (c_3)	1/3	2/3	0/3	0/3	0/3	0/3	3/3	0/3	3/3

Table 6.3: Table of the conditional probabilities $P(f_i|c_k)$

Note: The conditional probabilities are calculated as follows:

$$\begin{aligned}
 P((F_1 = \text{Slow})|c_1) &= \frac{\text{No. of records with } F_1 = \text{Slow and class label } c_1}{\text{No. of records with class label } c_1} \\
 &= 2/5.
 \end{aligned}$$

Step 3. We now calculate the following numbers:

$$\begin{aligned} q_1 &= P(x_1|c_1)P(x_2|c_1)P(x_3|c_1)P(c_1) \\ &= (2/5) \times (1/5) \times (3/5) \times (5/12) \\ &= 0.02 \end{aligned}$$

$$\begin{aligned} q_2 &= P(x_1|c_2)P(x_2|c_2)P(x_3|c_2)P(c_2) \\ &= (0/4) \times (0/4) \times (3/4) \times (4/12) \\ &= 0 \end{aligned}$$

$$\begin{aligned} q_3 &= P(x_1|c_3)P(x_2|c_3)P(x_3|c_3)P(c_3) \\ &= (2/3) \times (0/3) \times (3/3) \times (3/12) \\ &= 0 \end{aligned}$$

Step 4. Now

$$\max\{q_1, q_2, q_3\} = 0.05.$$

Step 5. The maximum is q_1 and it corresponds to the class label

$$c_1 = \text{"Animal"}.$$

So we assign the class label "Animal" to the test instance "(Slow, Rarely, No)".

Problem 2

Class-labeled training tuples from the *AllElectronics* customer database.

<i>RID</i>	<i>age</i>	<i>income</i>	<i>student</i>	<i>credit_rating</i>	<i>Class: buys_computer</i>
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

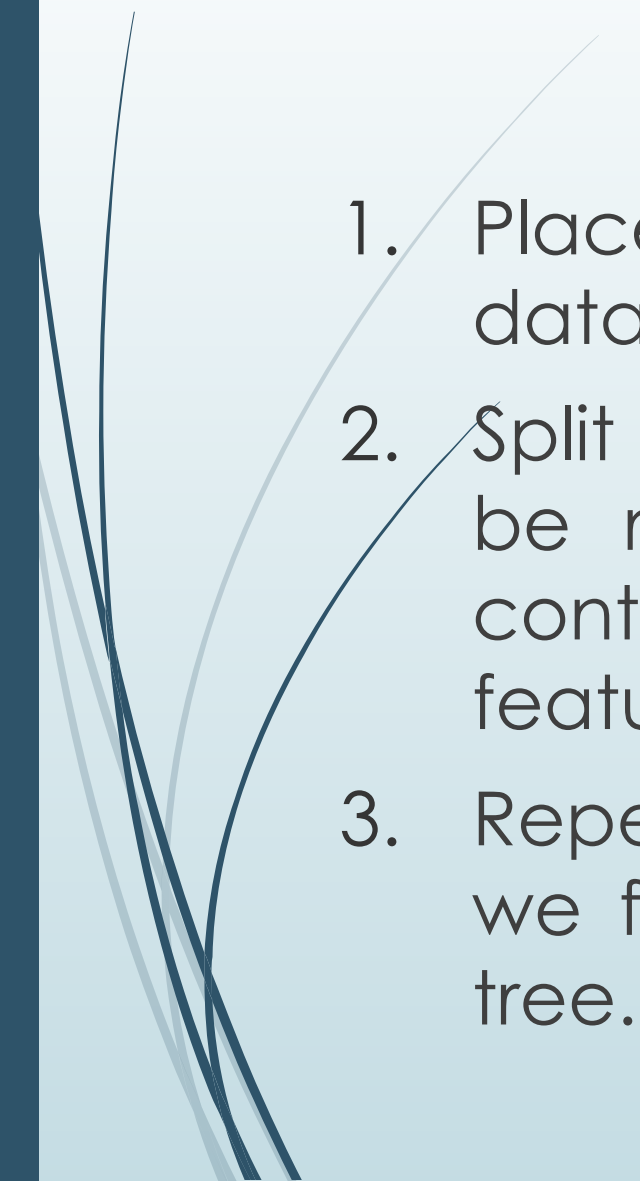
X = (age = youth, income = medium, student = yes, credit rating = fair)



DECISION TREE ALGORITHMS



Decision tree algorithm: Outline

- 
1. Place the “best” feature (or, attribute) of the dataset at the root of the tree.
 2. Split the training set into subsets. Subsets should be made in such a way that each subset contains data with the same value for a feature.
 3. Repeat Step 1 and Step 2 on each subset until we find leaf nodes in all the branches of the tree.



Some well known decision tree algorithms

1. ID3 (Iterative Dichotomiser 3) developed by Ross Quinlan
2. C4.5 developed by Ross Quinlan
3. C5.0 developed by Ross Quinlan
4. CART (Classification And Regression Trees)
5. 1R (One Rule) developed by Robert Holte in 1993.
6. RIPPER (Repeated Incremental Pruning to Produce Error Reduction) introduced in 1995 by William W. Cohen.



The ID3 algorithm

- ID3 stands for Iterative Dichotomiser 3
- Algorithm used to generate a decision tree.
- The ID3 algorithm was invented by Ross Quinlan.
- Dichotomiser means dividing into 2 completely opposite things.
- The ID3 follows the Occam's razor principle.
- Attempts to create the smallest possible decision tree.



Assumptions

- The algorithm uses information gain to select the most useful attribute for classification.
- We assume that there are only two class labels, namely, “+” and “-”. The examples with class labels “+” are called positive examples and others negative examples.

Notations

The following notations are used in the algorithm:

S	The set of examples
C	The set of class labels
F	The set of features
A	An arbitrary feature (attribute)
$\text{Values}(A)$	The set of values of the feature A
v	An arbitrary value of A
S_v	The set of examples with $A = v$
Root	The root node of a tree

Algorithm-ID3

Algorithm ID3(S, F, C)

1. Create a root node for the tree.
2. **if** (all examples in S are positive) **then**
3. **return** single node tree Root with label “+”
4. **end if**
5. **if** (all examples are negative) **then**
6. **return** single node tree Root with label “-”
7. **end if**
8. **if** (number of features is 0) **then**
9. **return** single node tree Root with label equal to the most common class label.
10. **else**
11. Let A be the feature in F with the highest information gain.
12. Assign A to the Root node in decision tree.
13. **for all** (values v of A) **do**
14. Add a new tree branch below Root corresponding to v .
15. **if** (S_v is empty) **then**
16. Below this branch add a leaf node with label equal to the most common class label in the set S .
17. **else**
18. Below this branch add the subtree formed by applying the same algorithm ID3 with the values ID3($S_v, C, F - \{A\}$).
19. **end if**
20. **end for**
21. **end if**

Example problem

Use ID3 algorithm to construct a decision tree for the data in Table 8.9.

Day	outlook	temperature	humidity	wind	playtennis
D1	sunny	hot	high	weak	no
D2	sunny	hot	high	strong	no
D3	overcast	hot	high	weak	yes
D4	rain	mild	high	weak	yes
D5	rain	cool	normal	weak	yes
D6	rain	cool	normal	strong	no
D7	overcast	cool	normal	strong	yes
D8	sunny	mild	high	weak	no
D9	sunny	cool	normal	weak	yes
D10	rain	mild	normal	weak	yes
D11	sunny	mild	normal	strong	yes
D12	overcast	mild	high	strong	yes
D13	overcast	hot	normal	weak	yes
D14	rain	mild	high	strong	no

Table 8.9: Training examples for the target concept “PlayTennis”

Solution

Root node

Table 8.9

Figure 8.7: Root node of the decision tree for data in Table 8.9

- **Step 1:** We first create a root node for the tree
- **Step 2:** Note that not all examples are positive (class label “yes”) and not all examples are negative (class label “no”). Also the number of features is not zero.
- **Step 3:** We have to decide which feature is to be placed at the root node.
 - For this, we have to calculate the information gains corresponding to each of the four features.

(i) Calculation of Entropy (S)

$$\begin{aligned}\text{Entropy}(S) &= -p_{\text{yes}} \log_2(p_{\text{yes}}) - p_{\text{no}} \log_2(p_{\text{no}}) \\ &= -(9/14) \times \log_2(9/14) - (5/14) \times \log_2(5/14) \\ &= 0.9405\end{aligned}$$

(ii) Calculation of Gain (S , outlook)

The values of the attribute “outlook” are “sunny”, “overcast” and “rain”. We have to calculate Entropy (S_v) for $v = \text{sunny}$, $v = \text{overcast}$ and $v = \text{rain}$.

$$\begin{aligned}\text{Entropy}(S_{\text{sunny}}) &= -(3/5) \times \log_2(3/5) - (2/5) \times \log_2(2/5) \\ &= 0.9710\end{aligned}$$

$$\begin{aligned}\text{Entropy}(S_{\text{overcast}}) &= -(4/4) \times \log_2(4/4) \\ &= 0\end{aligned}$$

$$\begin{aligned}\text{Entropy}(S_{\text{rain}}) &= -(3/5) \times \log_2(3/5) - (2/5) \times \log_2(2/5) \\ &= 0.9710\end{aligned}$$

$$\begin{aligned}\text{Gain}(S, \text{outlook}) &= \text{Entropy}(S) - \frac{|S_{\text{sunny}}|}{|S|} \times \text{Entropy}(S_{\text{sunny}}) \\ &\quad - \frac{|S_{\text{overcast}}|}{|S|} \times \text{Entropy}(S_{\text{overcast}}) \\ &\quad - \frac{|S_{\text{rain}}|}{|S|} \times \text{Entropy}(S_{\text{rain}}) \\ &= 0.9405 - (5/14) \times 0.9710 - (4/14) \times 0 \\ &\quad - (5/14) \times 0.9710 \\ &= 0.2469\end{aligned}$$

(iii) Calculation of Gain (S , temperature)

The values of the attribute “temperature” are “hot”, “mild” and “cool”. We have to calculate Entropy (S_v) for $v = \text{hot}$, $v = \text{mild}$ and $v = \text{cool}$.

$$\begin{aligned}\text{Entropy}(S_{\text{hot}}) &= -(2/4) \times \log_2(2/4) - (2/4) \times \log_2(2/4) \\ &= 1.0000\end{aligned}$$

$$\begin{aligned}\text{Entropy}(S_{\text{mild}}) &= -(4/6) \times \log_2(4/6) - (2/6) \times \log_2(2/6) \\ &= 0.9186\end{aligned}$$

$$\begin{aligned}\text{Entropy}(S_{\text{cool}}) &= -(3/4) \times \log_2(3/4) - (1/4) \times \log_2(1/4) \\ &= 0.8113\end{aligned}$$

$$\begin{aligned}\text{Gain}(S, \text{temperature}) &= \text{Entropy}(S) - \frac{|S_{\text{hot}}|}{|S|} \times \text{Entropy}(S_{\text{hot}}) \\ &\quad - \frac{|S_{\text{mild}}|}{|S|} \times \text{Entropy}(S_{\text{mild}}) \\ &\quad - \frac{|S_{\text{cool}}|}{|S|} \times \text{Entropy}(S_{\text{cool}}) \\ &= 0.9405 - (4/14) \times 1.0000 - (6/14) \times 0.9186 \\ &\quad - (4/14) \times 0.8113 \\ &= 0.0293\end{aligned}$$

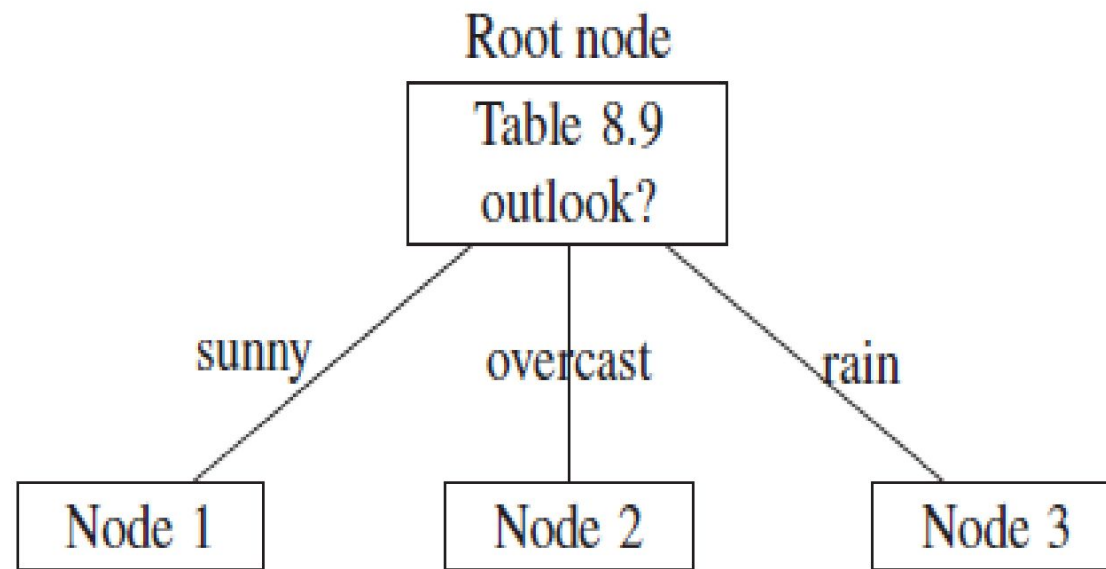
(iv) Calculation of $\text{Gain}(S, \text{humidity})$ and $\text{Gain}(S, \text{wind})$

The following information gains can be calculated in a similar way:

$$\text{Gain}(S, \text{humidity}) = 0.151$$

$$\text{Gain}(S, \text{wind}) = 0.048$$

- Step 4: We find the highest information gain which is the maximum among $\text{Gain}(S, \text{outlook})$, $\text{Gain}(S, \text{temperature})$, $\text{Gain}(S, \text{humidity})$ and $\text{Gain}(S, \text{wind})$.
- highest information gain = $\max\{0.2469, 0.0293, 0.151, 0.048\}$
 - $= 0.2469$
- This corresponds to the feature “outlook”. Therefore, we place “outlook” at the root node.



□ Step 5

Let $S^{(1)} = S_{\text{outlook}=\text{sunny}}$. We have $|S^{(1)}| = 5$. The examples in $S^{(1)}$ are shown in Table 8.10.

Day	outlook	temperature	humidity	wind	playtennis
D1	sunny	hot	high	weak	no
D2	sunny	hot	high	strong	no
D8	sunny	mild	high	weak	no
D9	sunny	cool	normal	weak	yes
D11	sunny	mild	normal	strong	yes

Table 8.10: Training examples with outlook = “sunny”

$$\begin{aligned}
\text{Gain}(S^{(1)}, \text{temp}) &= \text{Entropy}(S^{(1)}) - \frac{|S_{\text{temp} = \text{hot}}^{(1)}|}{|S^{(1)}|} \times \text{Entropy}(S_{\text{temp} = \text{hot}}^{(1)}) \\
&\quad - \frac{|S_{\text{temp} = \text{mild}}^{(1)}|}{|S^{(1)}|} \times \text{Entropy}(S_{\text{temp} = \text{mild}}^{(1)}) \\
&\quad - \frac{|S_{\text{temp} = \text{cool}}^{(1)}|}{|S^{(1)}|} \times \text{Entropy}(S_{\text{temp} = \text{cool}}^{(1)}) \\
&= \left[-(2/5) \log_2(2/5) - (3/5) \log_2(3/5) \right] \\
&\quad - (2/5) \times \left[-(2/2) \log(2/2) \right] \\
&\quad - (2/5) \times \left[-(1/2) \log(1/2) - (1/2) \log_2(1/2) \right] \\
&\quad - (1/5) \times \left[-(1/1) \log(1/1) \right] \\
&= 0.5709
\end{aligned}$$

$$\begin{aligned}
 \text{Gain}(S^{(1)}, \text{hum}) &= \text{Entropy}(S^{(1)}) - \frac{|S_{\text{hum}=\text{high}}^{(1)}|}{|S^{(1)}|} \times \text{Entropy}(S_{\text{hum}=\text{high}}^{(1)}) \\
 &\quad - \frac{|S_{\text{hum}=\text{normal}}^{(1)}|}{|S^{(1)}|} \times \text{Entropy}(S_{\text{hum}=\text{normal}}^{(1)}) \\
 &= \left[-(2/5) \log_2(2/5) - (3/5) \log_2(3/5) \right] \\
 &\quad - (3/5) \times \left[-(3/3) \log(3/3) \right] \\
 &\quad - (2/5) \times \left[-(2/2) \log(2/2) \right] \\
 &= 0.9709
 \end{aligned}$$

$$\begin{aligned}
 \text{Gain}(S^{(1)}, \text{wind}) &= \text{Entropy}(S^{(1)}) - \frac{|S_{\text{wind}=\text{weak}}^{(1)}|}{|S^{(1)}|} \times \text{Entropy}(S_{\text{wind}=\text{weak}}^{(1)}) \\
 &\quad - \frac{|S_{\text{wind}=\text{strong}}^{(1)}|}{|S^{(1)}|} \times \text{Entropy}(S_{\text{wind}=\text{strong}}^{(1)}) \\
 &= \left[-(2/5) \log_2(2/5) - (3/5) \log_2(3/5) \right] \\
 &\quad - (3/5) \times \left[-(2/3) \log(2/3) - (1/3) \log_2(1/3) \right] \\
 &\quad - (2/5) \times \left[-(1/2) \log(1/2) - (1/2) \log(1/2) \right] \\
 &= 0.0110
 \end{aligned}$$

The maximum of $\text{Gain}(S^{(1)}, \text{temp})$, $\text{Gain}(S^{(1)}, \text{hum})$ and $\text{Gain}(S^{(1)}, \text{wind})$ is $\text{Gain}(S^{(1)}, \text{hum})$. Hence we place “humidity” at Node 1 and split this node into two branches according to the values of the feature “humidity” to get the tree in Figure 8.9.

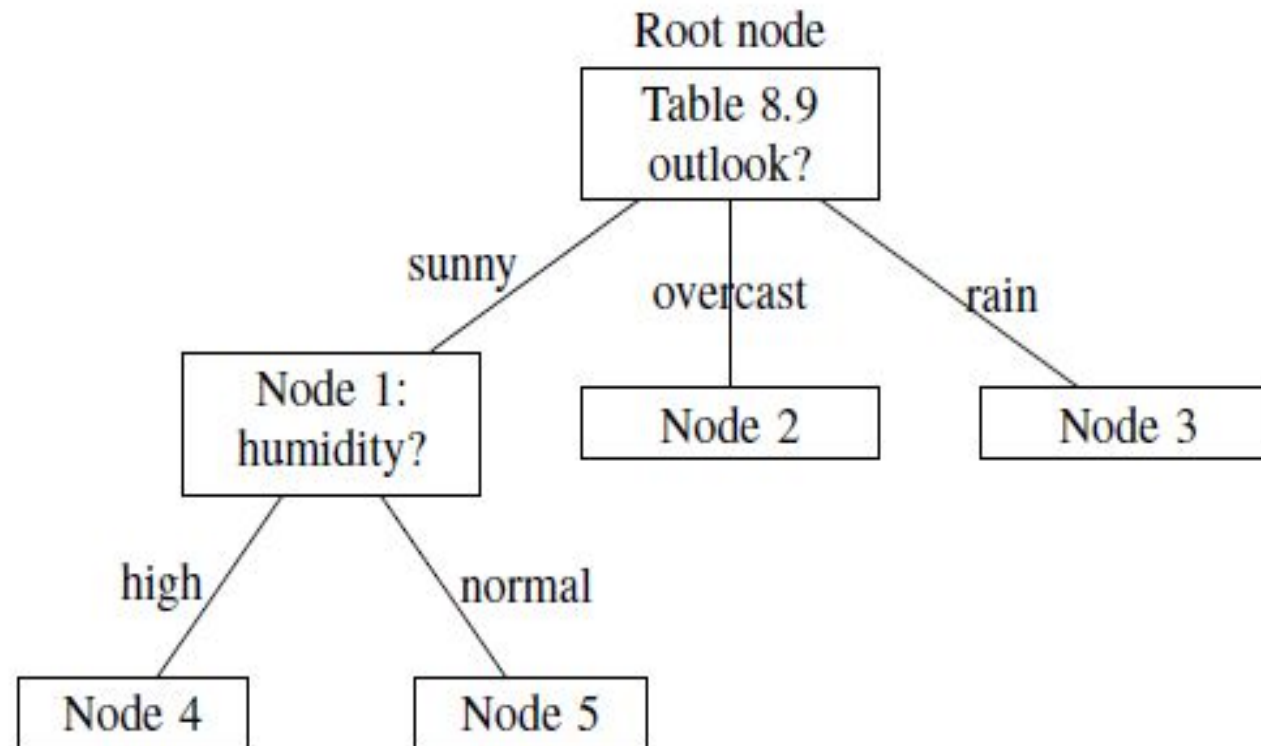


Figure 8.9: Decision tree for data in Table 8.9, after selecting the branching feature at Node 1

□ Step 6

It can be seen that all the examples in the the data set corresponding to Node 4 in Figure 8.9 have the same class label “no” and all the examples corresponding to Node 5 have the same class label “yes”. So we represent Node 4 as a leaf node with value “no” and Node 5 as a leaf node with value “yes”. Similarly, all the examples corresponding to Node 2 have the same class label “yes”. So we convert Node 2 as a leaf node with value “ yes. Finally, let $S^{(2)} = S_{\text{outlook} = \text{rain}}$.

The highest information gain for this data set is $G(S^{(2)}, \text{wind})$. All the examples corresponding to wind node have same class labels for each value.

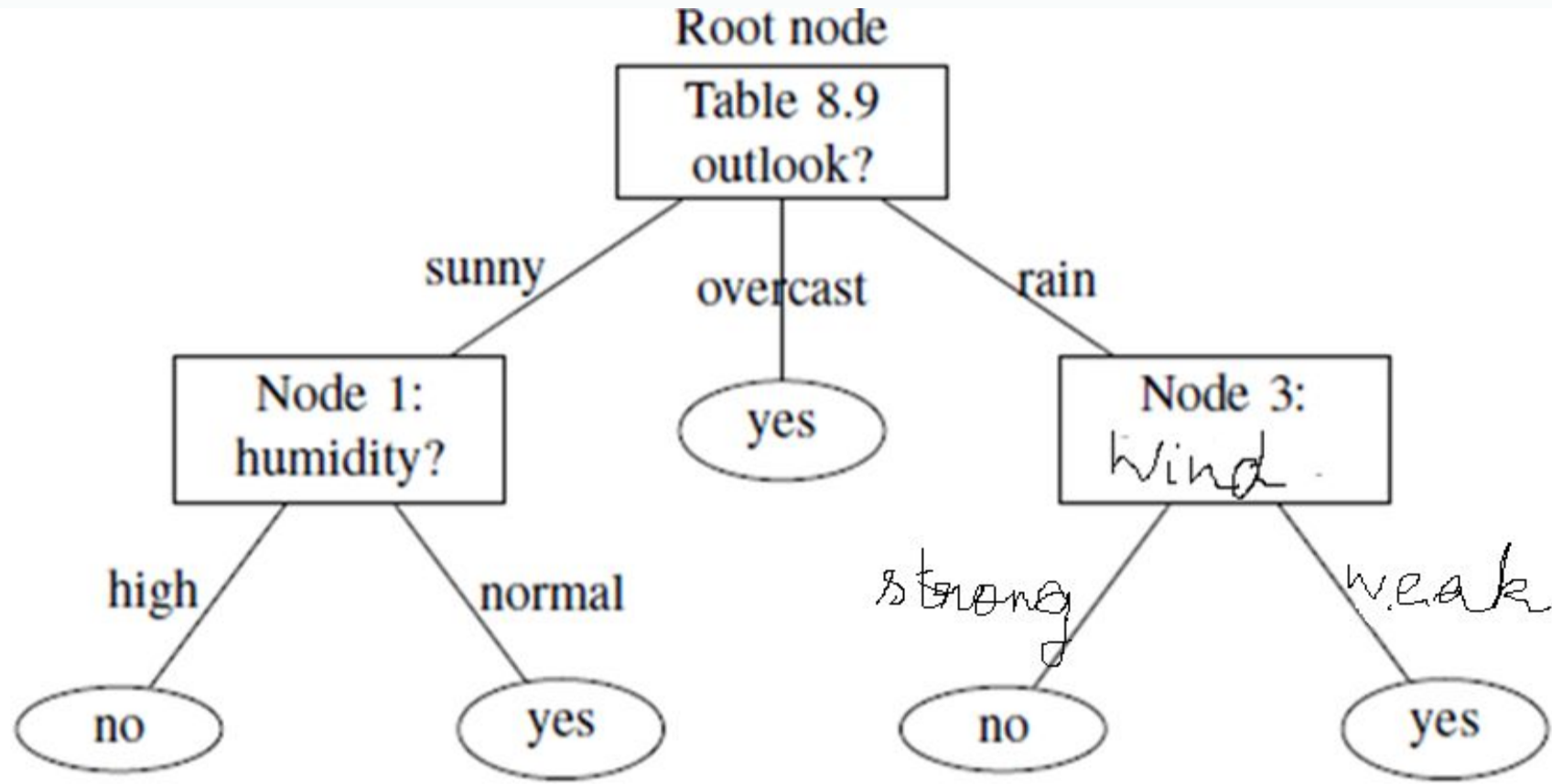


Figure 8.10: Decision tree for data in Table 8.9

Example dataset for information gain

<i>RID</i>	<i>age</i>	<i>income</i>	<i>student</i>	<i>credit_rating</i>	<i>Class: buys_computer</i>
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

class buys-computer $\rightarrow \{yes, no\}$

$$m = 2$$

$$C_1 = yes$$

$$C_2 = no$$

Information gain = $Info(D) - Info_A(D)$
expected information needed to clarify an instance in D .

$$Info(D) = - \sum_{i=1}^m p_i \cdot \log p_i \quad i = 1, 2$$
$$= - \frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right)$$
$$= 0.940 \text{ bits}$$

Next we need to compute the expected information requirement for each attribute

Start with attribute age

youth	muddle aged	senior
$Y = 2$	$Y = 4$	$Y = 3$
$N = 3$	$N = 0$	$N = 2$

Attribute age $\rightarrow$ $a_1: Y$ $a_2: M$ $a_3: S$
 $|D_1| = 5$ $|D_2| = 4$ $|D_3| = 5$

$$Info_{age}(D) = \frac{5}{14} info(D_1) + \frac{4}{14} info(D_2) + \frac{5}{14} info(D_3)$$

$$info(D_1) = - \frac{2}{5} \log_2 \left(\frac{2}{5} \right) - \frac{3}{5} \log_2 \left(\frac{3}{5} \right)$$
$$= \underline{\underline{0.970}}$$

$$\text{info}(D_2) = -\frac{4}{4} \log(4/4) - 0 = \underline{\underline{0}}$$

$$\text{info}(D_3) = -\frac{3}{5} \log(3/5) - 2/5 \log(2/5) = \underline{\underline{0.970}}$$

$$\begin{aligned} \text{Info}_{\text{age}}(D) &= \frac{5}{14} \times 0.97 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.97 \\ &= \underline{\underline{0.694}} \end{aligned}$$

$$\begin{aligned} \text{Gain}_{\text{age}} &= \text{Info}(D) - \text{Info}_{\text{age}}(D) \\ &= 0.940 - 0.694 = \underline{\underline{0.246}} \end{aligned}$$

Attribute income

$$\begin{array}{lll} a_1: H & a_2: M & a_3: L \\ |D_1| = 4 & |D_2| = 6 & |D_3| = 4 \end{array}$$

$$\begin{aligned} \text{info}_{\text{income}}(D) &= \frac{4}{14} \text{info}(D_1) + \frac{6}{14} \text{info}(D_2) \\ &\quad + \frac{4}{14} \text{info}(D_3) \end{aligned}$$

$$\text{info}(D_1) = -\frac{2}{4} \log(2/4) - 2/4 \log(2/4) = \underline{\underline{1}}$$

$$\begin{aligned} \text{info}(D_2) &= -\frac{4}{6} \log(4/6) - \frac{2}{6} \log(2/6) \\ &= \underline{\underline{0.918}} \end{aligned}$$

$$\begin{aligned} \text{info}(D_3) &= -\frac{3}{4} \log(3/4) - 1/4 \log(1/4) \\ &= \underline{\underline{0.811}} \end{aligned}$$

$$\begin{aligned} \text{info}_{\text{income}}(D) &= \frac{4}{14} \times 1 + \frac{6}{14} \times 0.918 + \frac{4}{14} \times 0.811 \\ &= \underline{\underline{0.91}} \end{aligned}$$

Attribute Student

$$a_1 = Y \quad a_2 = N$$

$$|D_1| = 7$$

$$|D_2| = 7$$

$$\text{info}_{\text{student}}(D) = \frac{7}{14} \text{info}(D_1) + \frac{7}{14} \text{info}(D_2)$$

$$\text{info}(D_1) = -\frac{6}{7} \log_2(6/7) - \frac{1}{7} \log_2(1/7) = \underline{0.59}$$

$$\text{info}(D_2) = -\frac{3}{7} \log_2(3/7) - \frac{4}{7} \log_2(4/7) = \underline{0.98}$$

$$\text{info}_{\text{student}}(D) = \frac{7}{14} \times 0.59 + \frac{7}{14} \times 0.98 = \underline{0.785}$$

Attribute CR

$$a_1 = F \quad a_2 = E$$

$$|D_1| = 8$$

$$|D_2| = 6$$

$$\text{info}_{\text{credit rating}}(D) = \frac{8}{14} \text{info}(D_1) + \frac{6}{14} \text{info}(D_2)$$

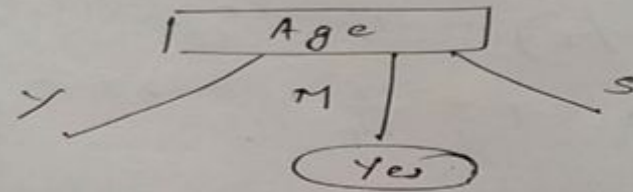
$$\text{info}(D_1) = -\frac{6}{8} \log_2(6/8) - \frac{2}{8} \log_2(2/8) = \underline{0.811}$$

$$\text{info}(D_2) = -\frac{6}{6} \log_2(6/6) - \frac{0}{6} \log_2(0/6) = \underline{0.98}$$

$$\text{info}_2(D_2) = -\frac{3}{6} \log_2(3/6) - \frac{3}{6} \log_2(3/6) = \underline{1}$$

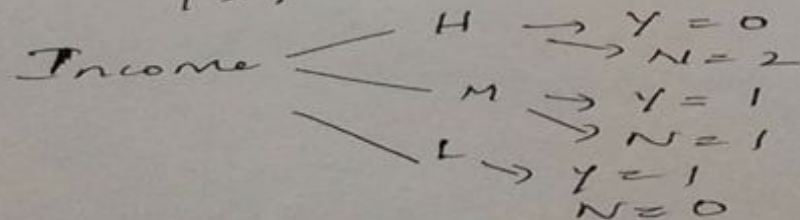
$$\text{info}_{\text{CR}}(D) = \frac{8}{14} \times 0.811 + \frac{6}{14} \times 1 = \underline{0.892}$$

	<u>Information before split</u>	<u>Information after split</u>	<u>Gain</u>
Age	0.94	0.692	0.248
Income	0.94	0.91	0.03
Student	0.94	0.785	0.155
CR	0.94	0.892	0.048



<u>Age</u>	<u>Income</u>	<u>Student</u>	<u>CR</u>	<u>class</u> <u>buys-comp</u>
Y	H	N	F	N
Y	H	N	E	N
Y	M	N	F	N
Y	L	Y	F	Y
Y	M	Y	E	Y

$$\begin{aligned}
 \frac{\text{Info}_{\text{Income}}(D)}{\text{Info}_{\text{Income}}} &= \sum_j \frac{|D_j|}{|D|} \ln \frac{|D|}{|D_j|} \\
 &= \frac{|D_1|}{|D|} \ln \frac{|D|}{|D_1|} + \frac{|D_2|}{|D|} \ln \frac{|D|}{|D_2|} + \frac{|D_3|}{|D|} \ln \frac{|D|}{|D_3|}
 \end{aligned}$$



$$= \frac{2}{5} \text{info}(D_1) + \frac{2}{5} \text{info}(D_2) + \frac{1}{5} \text{info}(D_3)$$

$$\text{info}(D_1) = 0 - \frac{2}{2} \log\left(\frac{2}{2}\right) = \underline{0}$$

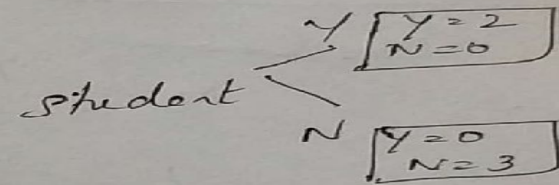
$$\text{info}(D_2) = -\frac{1}{2} \log\left(\frac{1}{2}\right) - \frac{1}{2} \log\left(\frac{1}{2}\right) = \underline{1}$$

$$\text{info}(D_3) = -\frac{1}{1} \log\left(\frac{1}{1}\right) - 0 = \underline{0}$$

$$\text{info}_{\text{income}}(D) = \frac{2}{5} \times 1 = \underline{0.4}$$

Attribute student

$$\text{info}_{\text{student}}(D) = \frac{2}{5} \text{info}(D_1) + \frac{3}{5} \text{info}(D_2)$$



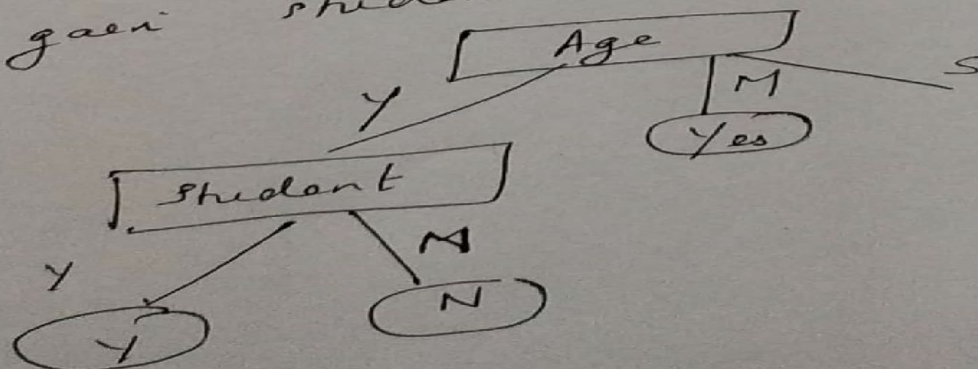
~~info student~~

$$\text{info}(D_1) = -\frac{2}{2} \log\left(\frac{2}{2}\right) - 0 = \underline{0}$$

$$\text{info}(D_2) = 0 - \frac{3}{3} \log\left(\frac{3}{3}\right) = \underline{0}$$

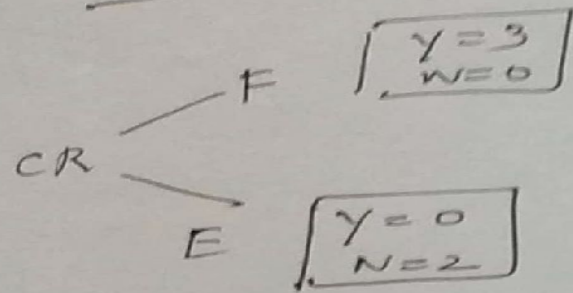
$$\text{info}_{\text{student}}(D) = \underline{0}$$

Age
Highest gain student



<u>Age</u>	<u>Income</u>	<u>CR</u>	<u>class buys_comp</u>
S	M	F	Y
S	L	F	Y
S	L	E	N
S	M	F	Y
S	M	E	N

Attribute CR

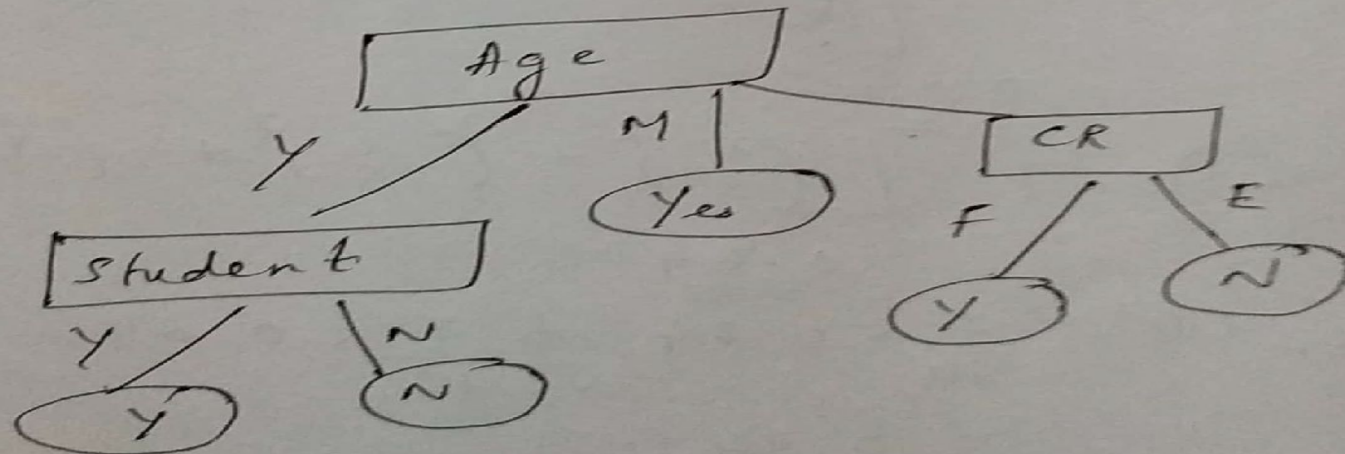


$$\text{info}_{CR}(D) = \frac{3}{5} \text{info}(D_1) + \frac{2}{5} \text{info}(D_2)$$

$$\text{info}(D_1) = -\frac{3}{3} \log \frac{3}{3} - 0 = \underline{\underline{0}}$$

$$\text{info}(D_2) = -\frac{2}{2} \log \left(\frac{2}{2}\right) = 0$$

$$\text{info}_{CR}(D) = 0$$



□ Generalisation and Overfitting -Idea of Overfitting, LASSO and RIDGE Regularization .

□ Idea of Training ,Testing ,validation

Generalization

•Objective of the machine learning system is to be able to make good prediction on new data. This is called generalization.

•Generalization in machine learning refers to the ability of a trained model to accurately make predictions on new, unseen data.

•The purpose of generalization is to equip the model to understand the patterns and relationships within its training data and apply them to previously unseen examples from within the same distribution as the training set.

•To achieve successful generalization, machine learning practitioners must address challenges like overfitting and underfitting.

Overfitting in Machine Learning

In the real world, the dataset present will never be clean and perfect. It means each dataset contains impurities, noisy data, outliers, missing data, or imbalanced data. Due to these impurities, different problems occur that affect the accuracy and the performance of the model. One of such problems is Overfitting in Machine Learning. *Overfitting is a problem that a model can exhibit.*

A statistical model is said to be overfitted if it can't generalize well with unseen data. Before understanding overfitting, we need to know some basic terms, which are:

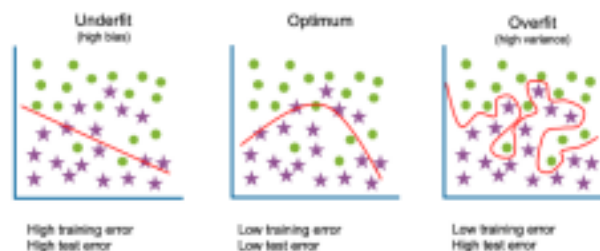
Noise: Noise is meaningless or irrelevant data present in the dataset. It affects the performance of the model if it is not removed.

Bias: Bias is a prediction error that is introduced in the model due to oversimplifying the machine learning algorithms. Or it is the difference between the predicted values and the actual values.

Variance: If the machine learning model performs well with the training dataset, but does not perform well with the test dataset, then variance occurs.

Generalization: It shows how well a model is trained to predict unseen data.

What is Overfitting?



- Overfitting & underfitting are the two main errors/problems in the machine learning model, which cause poor performance in Machine Learning.
- Overfitting occurs when the model fits more data than required, and it tries to capture each and every datapoint fed to it. Hence it starts capturing noise and inaccurate data from the dataset, which degrades the performance of the model.

- An overfitted model doesn't perform accurately with the test/unseen dataset and can't generalize well.
- An overfitted model is said to have low bias and high variance.

Example

We can understand overfitting with a general example. Suppose there are three students, X, Y, and Z, and all three are preparing for an exam. X has studied only three sections of the book and left all other sections. Y has a good memory, hence memorized the whole book. And the third student, Z, has studied and practiced all the questions. So, in the exam, X will only be able to solve the questions if the exam has questions related to section 3. Student Y will only be able to solve questions if they appear exactly the same as given in the book. Student Z will be able to solve all the exam questions in a proper way.

The same happens with machine learning; if the algorithm learns from a small part of the data, it is unable to capture the required data points and hence under fitted.

Suppose the model learns the training dataset, like the Y student. They perform very well on the seen dataset but perform badly on unseen data or unknown instances. In such cases, the model is said to be Overfitting.

And if the model performs well with the training dataset and also with the test/unseen dataset, similar to student Z, it is said to be a good fit.

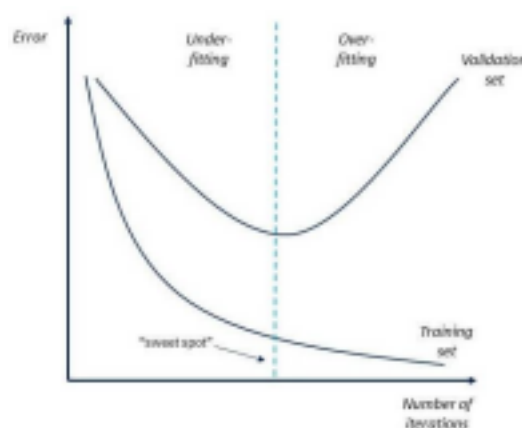
How to detect Overfitting?

Overfitting in the model can only be detected once you test the data. To detect the issue, we can perform Train/test split.

In the train-test split of the dataset, we can divide our dataset into random test and training datasets. We train the model with a training dataset which is about 80% of the total dataset. After training the model, we test it with the test dataset, which is 20 % of the total dataset.

Now, if the model performs well with the training dataset but not with the test dataset, then it is likely to have an overfitting issue.

For example, if the model shows 85% accuracy with training data and 50% accuracy with the test dataset, it means the model is not performing well.



Ways to prevent the Overfitting

Although overfitting is an error in Machine learning which reduces the performance of the model, however, we can prevent it in several ways. With the use of the linear model, we can avoid overfitting; however, many real-world problems are non-linear ones. It is important to prevent overfitting from the models. Below are several ways that can be used to prevent overfitting:

1. Early Stopping

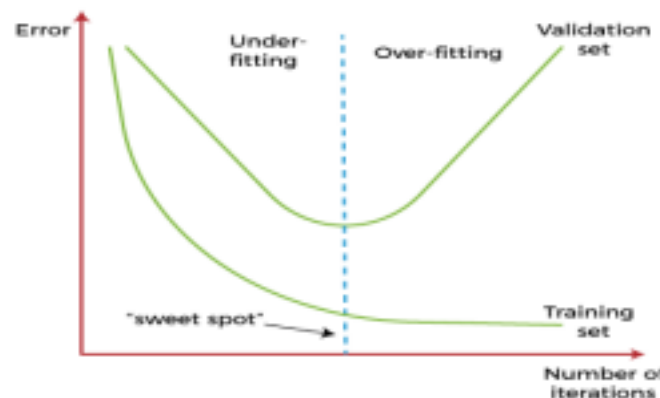
2. Train with more data
3. Feature Selection
4. Cross-Validation
5. Data Augmentation
6. Regularization

Early Stopping

In this technique, the training is paused before the model starts learning the noise within the model. In this process, while training the model iteratively, measure the performance of the model after each iteration. Continue up to a certain number of iterations until a new iteration improves the performance of the model.

After that point, the model begins to overfit the training data; hence we need to stop the process before the learner passes that point.

Stopping the training process before the model starts capturing noise from the data is known as early stopping.



However, this technique may lead to the underfitting problem if training is paused too early. So, it is very important to find that "sweet spot" between underfitting and overfitting.

Train with More data

Increasing the training set by including more data can enhance the accuracy of the model, as it provides more chances to discover the relationship between input and output variables. It may not always work to prevent overfitting, but this way helps the algorithm to detect the signal better to minimize the errors.

When a model is fed with more training data, it will be unable to overfit all the samples of data and forced to generalize well.

But in some cases, the additional data may add more noise to the model; hence we need to be sure that data is clean and free from in-consistencies before feeding it to the model.

Feature Selection

While building the ML model, we have a number of parameters or features that are used to predict the outcome. However, sometimes some of these features are redundant or less important for the prediction, and for this feature selection process is applied. In the feature selection process, we identify the most important features within training data, and other features are removed. Further, this process helps to simplify the model and reduces noise from the data. Some algorithms have the auto-feature selection, and if not, then we can manually perform this process.

Cross-Validation

Cross-validation is one of the powerful techniques to prevent overfitting.

In the general k-fold cross-validation technique, we divided the dataset into k-equal-sized subsets of data; these subsets are known as folds.

Data Augmentation

Data Augmentation is a data analysis technique, which is an alternative to adding more data to prevent overfitting. In this technique, instead of adding more training data, slightly modified copies of already existing data are added to the dataset.

The data augmentation technique makes it possible to appear data sample slightly different every time it is processed by the model. Hence each data set appears unique to the model and prevents overfitting.

Regularization

If overfitting occurs when a model is complex, we can reduce the number of features. However, overfitting may also occur with a simpler model, more specifically the Linear model, and for such cases, regularization techniques are much helpful.

Regularization is the most popular technique to prevent overfitting. It is a group of methods that forces the learning algorithms to make a model simpler. Applying the regularization technique may slightly increase the bias but slightly reduces the variance. In this technique, we modify the objective function by adding the penalizing term, which has a higher value with a more complex model.

The two commonly used regularization techniques are L1 Regularization and L2

Regularization. Ensemble Methods

In ensemble methods, prediction from different machine learning models is combined to identify the most popular result.

The most commonly used ensemble methods are Bagging and Boosting.

In bagging, individual data points can be selected more than once. After the collection of several sample datasets, these models are trained independently, and depending on the type of task-i.e., regression or classification-the average of those predictions is used to predict a more accurate result. Moreover, bagging reduces the chances of overfitting in complex models.

In boosting, a large number of weak learners arranged in a sequence are trained in such a way that each learner in the sequence learns from the mistakes of the learner before it. It combines all the weak learners to come out with one strong learner. In addition, it improves the predictive flexibility of simple models.

Regularization in Machine Learning

Regularization is one of the most important concepts of machine learning. It is a technique to prevent the model from overfitting by adding extra information to it.

Sometimes the machine learning model performs well with the training data but does not perform well with the test data. It means the model is not able to predict the output when deals with unseen data by introducing noise in the output, and hence the model is called overfitted. This problem can be deal with the help of a regularization technique.

This technique can be used in such a way that it will allow to maintain all variables or features in the model by reducing the magnitude of the variables. Hence, it maintains accuracy as well as a generalization of the model.

It mainly regularizes or reduces the coefficient of features toward zero. In simple words, "*In regularization technique, we reduce the magnitude of the features by keeping the same number of features.*"

How does Regularization Work?

Regularization works by adding a penalty or complexity term to the complex model. Let's consider the simple linear regression equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_n x_n + b$$

In the above equation, Y represents the value to be predicted

X1, X2, ...Xn are the features for Y.

$\beta_0, \beta_1, \dots, \beta_n$ are the weights or magnitude attached to the features, respectively. Here represents the bias of the model, and b represents the intercept.

Linear regression models try to optimize the β_0 and b to minimize the cost function. The equation for the cost function for the linear model is given below:

$$\sum_{i=1}^M (y_i - y'_i)^2 = \sum_{i=1}^M (y_i - \sum_{j=0}^n \beta_j * X_{ij})^2$$

Now, we will add a loss function and optimize parameter to make the model that can predict the accurate value of Y . The loss function for the linear regression is called as RSS or Residual sum of squares.

Techniques of Regularization

There are mainly two types of regularization techniques, which are given

below: ○ Ridge Regression

○ Lasso Regression

Ridge Regression

- Ridge regression is one of the types of linear regression in which a small amount of bias is introduced so that we can get better long-term predictions.
- Ridge regression is a regularization technique, which is used to reduce the complexity of the model. It is also called as L2 regularization.
- In this technique, the cost function is altered by adding the penalty term to it. The amount of bias added to the model is called Ridge Regression penalty. We can calculate it by multiplying with the lambda to the squared weight of each individual feature.
- The equation for the cost function in ridge regression will be:

$$\sum_{i=1}^M (y_i - y'_i)^2 = \sum_{i=1}^M \left(y_i - \sum_{j=0}^n \beta_j * x_{ij} \right)^2 + \lambda \sum_{j=0}^n \beta_j^2$$

- In the above equation, the penalty term regularizes the coefficients of the model, and hence ridge regression reduces the amplitudes of the coefficients that decreases the complexity of the model.
- As we can see from the above equation, if the values of λ tend to zero, the equation becomes the cost function of the linear regression model. Hence, for the minimum value of λ , the model will resemble the linear regression model.
- A general linear or polynomial regression will fail if there is high collinearity between the independent variables, so to solve such problems, Ridge regression can be used. ○ It helps to solve the problems if we have more parameters than samples.

Lasso Regression:

- Lasso regression is another regularization technique to reduce the complexity of the model. It stands for Least Absolute and Selection Operator.

- It is similar to the Ridge Regression except that the penalty term contains only the absolute weights instead of a square of weights.
- Since it takes absolute values, hence, it can shrink the slope to 0, whereas Ridge Regression can only shrink it near to 0.
- It is also called as L1 regularization. The equation for the cost function of Lasso regression will be:

$$\sum_{i=1}^M (y_i - y'_i)^2 = \sum_{i=1}^M \left(y_i - \sum_{j=0}^n \beta_j * x_{ij} \right)^2 + \lambda \sum_{j=0}^n |\beta_j|$$

- Some of the features in this technique are completely neglected for model evaluation. ○ Hence, the Lasso regression can help us to reduce the overfitting in the model as well as the feature selection.

Key Difference between Ridge Regression and Lasso Regression

- Ridge regression is mostly used to reduce the overfitting in the model, and it includes all the features present in the model. It reduces the complexity of the model by shrinking the coefficients.
- Lasso regression helps to reduce the overfitting in the model as well as feature selection.

Difference Between Ridge and Lasso

Feature	Ridge (L2)	Lasso (L1)
Penalty	Square of weights	Absolute weights
Feature Selection	No	Yes
Weight Reduction	Small values	Can become zero
Complexity	Reduces	Reduces strongly
Sparse Model	No	Yes

Idea of Training, Validation, and Testing

Machine Learning aims to build systems that can learn patterns from data and make predictions on unseen data.

To achieve this, the available dataset is divided into different parts:

1. Training Set
2. Validation Set
3. Testing Set

These datasets help in:

- Learning patterns
- Improving model performance
- Avoiding overfitting
- Evaluating generalization ability

Without proper dataset splitting, the model may memorize data instead of learning meaningful patterns.

Why Dataset Splitting is Necessary

Suppose a student studies only one set of questions and memorizes them.

If the same questions appear in the exam, the student scores well.

But if new questions appear, performance may become poor.

Similarly, a machine learning model may:

- Memorize training data
- Fail on unseen data

Therefore, separate datasets are required to properly evaluate the model.

Training Set

The training set is the portion of data used to train the machine learning model.

The model learns:

- Relationships between input and output
- Patterns in data
- Important features

Purpose of Training Data

The training set is used to:

- Learn model parameters
- Minimize prediction error
- Adjust weights and biases

Working of Training Phase

During training:

1. Input data is given to the model
2. Model predicts output
3. Error is calculated
4. Parameters are updated
5. Process repeats multiple times

This process is called **learning**.

Example

Suppose we want to predict house prices.

Inputs

- Area
- Number of rooms
- Location

Output

- House price

The model learns the relationship between inputs and output using training data.

Characteristics of Training Data

Feature	Description
Used for learning	Yes
Model updates weights	Yes
Largest portion of dataset	Yes
Seen repeatedly during training	Yes

Validation Set

The validation set is used to evaluate the model during training and tune hyperparameters.

The model does NOT learn directly from validation data.

Need for Validation Set

During training, different choices must be made such as:

- Learning rate
- Number of hidden layers
- Batch size
- Regularization parameter
- Number of epochs

These are called **hyperparameters**.

Validation data helps select the best hyperparameters.

Purpose of Validation Set

The validation set is used for:

- Hyperparameter tuning
- Model comparison
- Detecting overfitting
- Early stopping

Hyperparameters vs Parameters

Parameters	Hyperparameters
Learned automatically	Set manually
Example: weights	Example: learning rate
Updated during training	Chosen before training

Example of Validation Usage

Suppose three models are trained:

Model	Validation Accuracy
Model A	82%
Model B	91%
Model C	87%

Model B is selected because it performs best on validation data.

Early Stopping Using Validation Data

Sometimes:

- Training accuracy keeps increasing
- Validation accuracy starts decreasing

This indicates overfitting.

Training is stopped at the point where validation performance is best.

Characteristics of Validation Set

Feature	Description
Used during training	Yes
Used for learning weights	No

Used for tuning	Yes
Helps prevent overfitting	Yes

Testing Set

The testing set is used to evaluate the final performance of the trained model.

It represents unseen real-world data.

Purpose of Testing Set

Testing data is used to:

- Measure generalization ability
- Evaluate final model accuracy
- Estimate real-world performance

Important Rule About Testing Data

Testing data should:

- Never be used during training
- Never be used for tuning hyperparameters

Otherwise, evaluation becomes biased.

Characteristics of Testing Set

Feature	Description
Used after training	Yes
Used for learning	No
Used for tuning	No
Completely unseen data	Yes

Typical Dataset Splits

Common Splits

Training	Validation	Testing
70%	15%	15%
80%	10%	10%
60%	20%	20%

Choice depends on:

- Dataset size
- Application
- Model complexity

Complete Workflow

Complete Dataset



Split Data

Training	Validation	Testing		
----------	------------	---------	--	--



Train Model on Training Data



Tune Model using Validation Data



Select Best Model



Evaluate Final Performance on Testing Data

1. Classification Performance Evaluation Metrics: Accuracy, Precision, Precision, Recall, Specificity, False Positive Rate (FPR), F1 Score, Receiver Operator Characteristic (ROC) Curve, AUC.
2. Regression Performance Evaluation Metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R Squared/Coefficient of Determination

Introduction

How to evaluate the performance of a machine learning model?

Let us consider a task to classify whether a person is **pregnant** or **not pregnant**.

If the test for pregnancy is positive (+ve), then the person is pregnant. On the other hand, if the test for pregnancy is negative (-ve) then the person is not pregnant.

Now consider the above classification (pregnant or not pregnant) carried out by a machine learning algorithm. The output of the machine learning algorithm can be mapped to one of the following categories.

1. A person who is actually pregnant (positive) and classified as pregnant (positive). This is called **TRUE POSITIVE (TP)**.



Figure 1: True Positive.

2. A person who is actually not pregnant (negative) and classified as not pregnant (negative). This is called **TRUE NEGATIVE (TN)**.



Figure 2: True Negative.

3. A person who is actually not pregnant (negative) and classified as pregnant (positive). This is called **FALSE POSITIVE (FP)**.



Figure 3: False Positive.

4. A person who is actually pregnant (positive) and classified as not pregnant (negative). This is called **FALSE NEGATIVE (FN)**.



Figure 4: False Negative.

☐ **True Positive (TP): predicted positive and it's true**

Eg:- A person who is actually pregnant (positive) and classified as pregnant (positive). This is called **TRUE POSITIVE (TP)**.

☐ **True Negative (TN): predicted negative and it's true.**

○ **Eg:-** A person who is actually not pregnant (negative) and classified as not pregnant (negative). This is called **TRUE NEGATIVE (TN)**.

☐ **False Positive (FP): (Type 1 Error) : predicted positive and it's false.**

○ **Eg:-** A person who is actually not pregnant (negative) and classified as pregnant (positive). This is called **FALSE POSITIVE (FP)**.

☐ **False Negative (FN): (Type 2 Error): predicted negative and it's false.**

○ **Eg:-** A person who is actually pregnant (positive) and classified as not pregnant (negative). This is called **FALSE NEGATIVE (FN)**.

Confusion Matrix

*In any machine learning model what we desire is **TRUE POSITIVE** and **TRUE NEGATIVE** but due to the misclassifications, we may also end up in **FALSE POSITIVE** and **FALSE NEGATIVE**.*

In the case of the pregnancy classification problem, there is confusion in classifying whether a person is pregnant or not. This is because no machine learning algorithm is perfect. This confusion in classifying the data in a matrix called confusion matrix.

- ❑ A confusion matrix is a table that is often used to describe the performance of a classification model (or "classifier") on a set of test data for which the true values are known.
- ❑ The confusion matrix itself is relatively simple to understand, but the related terminology can be confusing.
- ❑ Classification accuracy alone can be misleading if you have an unequal number of observations in each class or if you have more than two classes in your dataset.
- ❑ Calculating a confusion matrix can give you a better idea of what your classification model is getting right and what types of errors it is making.

Confusion Matrix of binary class

		Actual Values	
		1	0
Predicted Values	1	 TRUE POSITIVE	 FALSE POSITIVE TYPE 1 ERROR
	0	 FALSE NEGATIVE TYPE 2 ERROR	 TRUE NEGATIVE

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

The confusion matrix can also be represented as shown below. But predicted and actual values should be properly labeled.

If we consider a binary classification problem,
 C_{00} represents the count of true negative
 C_{01} represents the count of false positive
 C_{10} represents the count of false negative and
 C_{11} represents the count of true positive.

		PREDICTED LABEL	
		NEGATIVE	POSITIVE
TRUE LABEL	NEGATIVE	 TRUE NEGATIVE	 FALSE POSITIVE
	POSITIVE	 FALSE NEGATIVE	 TRUE POSITIVE

		PREDICTED LABEL	
		NEGATIVE	POSITIVE
TRUE LABEL	NEGATIVE	TRUE NEGATIVE	FALSE POSITIVE
	POSITIVE	FALSE NEGATIVE	TRUE POSITIVE

Confusion matrix of classes > 2

true class i	class j predicted by a classifier										
	'0'	'1'	'2'	'3'	'4'	'5'	'6'	'7'	'8'	'9'	'R'
'0'	97	0	0	0	0	0	1	0	0	1	1
'1'	0	98	0	0	1	0	0	1	0	0	0
'2'	0	0	96	1	0	1	0	1	0	0	1
'3'	0	0	2	95	0	1	0	0	1	0	1
'4'	0	0	0	0	98	0	0	0	0	2	0
'5'	0	0	0	1	0	97	0	0	0	0	2
'6'	1	0	0	0	0	1	98	0	0	0	0
'7'	0	0	1	0	0	0	0	98	0	0	1
'8'	0	0	0	1	0	0	1	0	96	1	1
'9'	1	0	0	0	3	1	0	0	0	95	0

		predicted	
		negative	positive
actual examples	negative	<i>a</i> TN - True Negative correct rejections	<i>b</i> FP - False Positive false alarms type I error
	positive	<i>c</i> FN - False Negative misses, type II error overlooked danger	<i>d</i> TP - True Positive hits

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Confusion matrix for Covid-19 Prediction

- ❑ 165 people were being tested for the presence of Covid-19 (i.e. The classifier made a total of 165 predictions).
- ❑ Out of those 165 cases, the classifier predicted "yes" 110 times, and "no" 55 times.
- ❑ In reality, 105 patients in the sample have the disease, and 60 patients do not.

n=165	Predicted:		
	NO	YES	
Actual: NO	TN = 50	FP = 10	60
Actual: YES	FN = 5	TP = 100	105
		55	110

Classification Performance Evaluation Metrics:

Accuracy

Accuracy is a metric that generally describes how the model performs across all classes. It is useful when all classes are of equal importance. It is calculated as the ratio between the number of correct predictions to the total number of predictions.

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

$$\text{Accuracy} = \frac{\text{True}_{\text{positive}} + \text{True}_{\text{negative}}}{\text{True}_{\text{positive}} + \text{True}_{\text{negative}} + \text{False}_{\text{positive}} + \text{False}_{\text{negative}}}$$

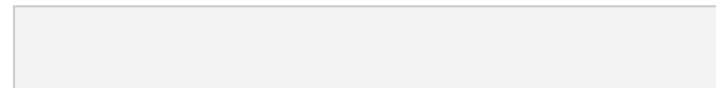
So it can be written as:

Example-1:

Calculate the accuracy of the following model that classified 100 tumors as malignant (the positive class) or benign (the negative class):

- True Positive (TP):
 - ML model predicted: Malignant •
 - Reality: Malignant
- Number of TP results: 1 False

Negative (FN):	Number of FP results: 1 True
• Reality: Malignant	Negative (TN):
• ML model predicted: Benign •	• Reality: Benign
Number of FN results: 8	• ML model predicted: Benign •
False Positive (FP):	Number of TN results: 90
• Reality: Benign	
• ML model predicted: Malignant •	



Accuracy comes out to 0.91, or 91% (91 correct predictions out of 100 total examples). That means our tumor classifier is doing a great job of identifying malignancies, right?

Actually, let's do a closer analysis of positives and negatives to gain more insight into our model's performance. Of the 100 tumor examples, 91 are benign (90 TNs and 1 FP) and 9 are malignant (1 TP and 8 FNs).

Example-2

Let us consider a task to classify whether a person is **pregnant** or **not pregnant**

Now, we select 100 people which includes pregnant women, not pregnant women and men with fat belly. Let us assume out of this 100 people 40 are pregnant and the remaining 60 people include not pregnant women and men with fat belly. We now use a machine learning algorithm to predict the outcome. The predicted outcome (pregnancy +ve or -ve) using a machine learning algorithm is termed as the ***predicted label*** and the true outcome (in this case which we know from doctor's/expert's record) is termed as the ***true label***.

Out of 40 pregnant women 30 pregnant women are classified correctly and the remaining 10 pregnant women are classified as not pregnant by the machine learning algorithm. On the other hand, out of 60 people in the not pregnant category, 55 are classified as not pregnant and the remaining 5 are classified as pregnant.

In this case, $TN = 55$, $FP = 5$, $FN = 10$, $TP = 30$. The confusion matrix is as follows.

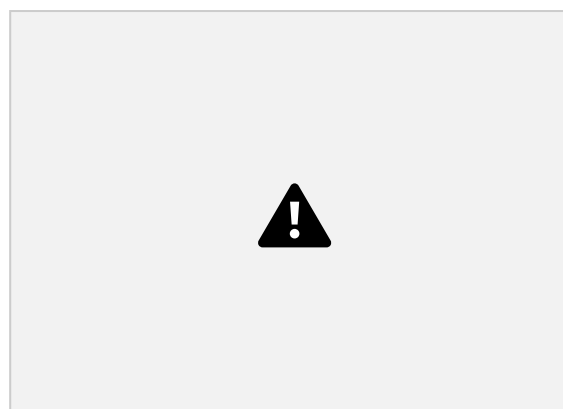
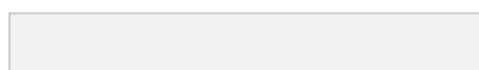


Figure: Confusion matrix for the pregnant vs not pregnant classification.

What is the accuracy of the machine learning model for this classification task?



Accuracy represents the number of correctly classified data instances over the total number of data instances. $Accuracy = (55 + 30) / (55 + 5 + 30 + 10) = 0.85$ and in percentage the accuracy will be 85%. Is

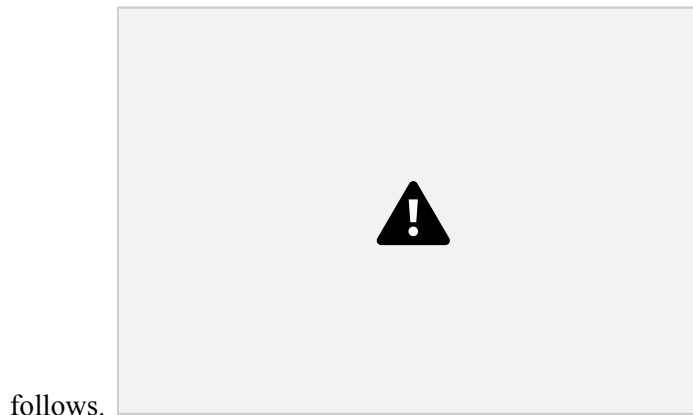
accuracy the best measure?

Accuracy may not be a good measure if the dataset is not balanced (both negative and positive classes have different number of data instances).

Example-3

Consider the following scenario: There are 90 people who are healthy (negative) and 10 people who have some disease (positive). Now let's say our machine learning model perfectly classified the 90 people as healthy but it also classified the unhealthy people as healthy. What will happen in this scenario? Let us see the confusion matrix and find out the accuracy?

In this example, $TN = 90$, $FP = 0$, $FN = 10$ and $TP = 0$. The confusion matrix is as



Confusion matrix for healthy vs unhealthy people classification task.

Accuracy in this case will be $(90 + 0)/(100) = 0.9$ and in percentage the accuracy is 90%. Is there anything fishy?

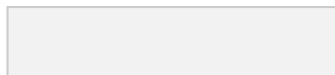
The accuracy in this case is 90 % but this model is very poor because all the 10 people who are unhealthy are classified as healthy. By this example what we are trying to say is that accuracy is not a good metric when the data set is unbalanced. Using accuracy in such scenarios can result in misleading interpretation of results.

So now we move further to find out another metric for classification.

Precision

Precision is the ratio of true positives to total predicted positives. The precision is calculated as the ratio between the number of *Positive* samples correctly classified to the total number of samples classified as *Positive* (either correctly or incorrectly).

The precision measures the model's accuracy in classifying a sample as positive. Precision is defined as follows:



What does precision mean?

Precision should ideally be 1 (high) for a good classifier. **Precision** becomes 1 only when the numerator and denominator are equal i.e $TP = TP + FP$, this also means **FP** is zero. As **FP** increases the value of denominator becomes greater than the numerator and **precision** value decreases (which we don't want).

The precision reflects how reliable the model is in classifying samples as Positive.

When the model makes many incorrect *Positive* classifications, or few correct *Positive* classifications, this increases the denominator and makes the precision small. On the other hand, the precision is high when:

1. The model makes many correct *Positive* classifications (maximize *True Positive*). 2.

The model makes fewer incorrect *Positive* classifications (minimize *False Positive*).

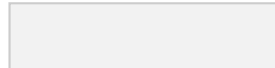
Imagine a man who is trusted by others; when he predicts something, others believe him. The precision is like this man. When the precision is high, you can trust the model when it predicts a sample as *Positive*. Thus, the precision helps to know how the model is accurate when it says that a sample is *Positive*.

So in the pregnancy example, **precision** = $30/(30+5) = 0.857$

Recall or Sensitivity

Recall is also known as **sensitivity** or **true positive rate**.

Recall or sensitivity is the ratio of true positives to total (actual) positives in the data. The recall or sensitivity is calculated as the ratio between the number of *Positive* samples correctly classified as *Positive* to the total number of *Positive* samples. The recall measures the model's ability to detect *Positive* samples. The higher the recall, the more positive samples detected. It is defined as follows



Recall should ideally be 1 (high) for a good classifier. **Recall** becomes 1 only when the numerator and denominator are equal i.e $TP = TP + FN$, this also means **FN** is zero. As **FN** increases the value of denominator becomes greater than the numerator and **recall** value decreases (which we don't want).

The recall cares only about how the positive samples are classified. This is independent of how the negative samples are classified, e.g. for the precision. When the model classifies all the positive samples as *Positive*, then the recall will be 100% even if all the negative samples were incorrectly classified as *Positive*.

- The precision measures the model trustiness in classifying positive samples, and the recall measures how many positive samples were correctly classified by the model.
- The precision takes into account how both the positive and negative samples were classified, but the recall only considers the positive samples in its calculations. In other words, the precision is dependent on both the negative and positive samples, but the recall is dependent only on the positive samples (and independent of the negative samples).
- The precision considers when a sample is classified as *Positive*, but it does not care about correctly classifying *all* positive samples. The recall cares about correctly classifying *all* positive samples, but it does not care if a negative sample is classified as positive.
- When a model has high recall but low precision, then the model classifies most of the positive samples correctly but it has many false positives (i.e. classifies many *Negative* samples as *Positive*). When a model has high precision but low recall, then the model is accurate when it classifies a sample as *Positive* but it can only classify a few positive samples.

So in the pregnancy example let us see what will be the recall.

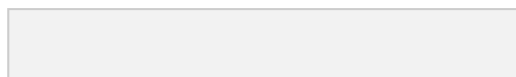
$$\text{Recall} = 30/(30+10) = 0.75$$

So ideally in a good classifier, we want both **precision** and **recall** to be one which also means **FP** and **FN** are zero. Therefore we need a metric that takes into account both **precision** and **recall**.

F1-score

F1-score is a metric which takes into account both **precision** and **recall** and is defined as

follows:



F1 Score becomes 1 only when **precision** and **recall** are both 1. **F1 score** becomes high only when both **precision** and **recall** are high. **F1 score** is the harmonic mean of **precision** and **recall** and is a better measure than **accuracy**.

In the pregnancy example, **F1 Score** = $2 * (0.857 * 0.75) / (0.857 + 0.75) = 0.799$.

Specificity

Specificity is the Ratio of true negatives to total negatives in the data.

Specificity is the correctly -ve labeled by the program to all who are healthy in reality.

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

- ☐ Error Rate OR Misclassification Rate: $(1 - \text{Accuracy})$ OR $(\text{FP} + \text{FN}) / \text{total}$
- ☐ True Positive Rate ("Sensitivity" or "Recall"): $\text{TP} / \text{actual yes}$
- ☐ False Positive Rate: $\text{FP} / \text{actual no}$
- ☐ True Negative Rate ("Specificity" OR $(1 - \text{False Positive Rate})$): $\text{TN} / \text{actual no}$
- ☐ Precision: $\text{TP} / \text{predicted yes}$
- ☐ Prevalence: $\text{actual yes} / \text{total}$

Summary of Measuring classifier performance

- ☐ **Accuracy:** $(\text{TP} + \text{TN}) / \text{total}$
- ☐ **Error Rate OR Misclassification Rate:** $(1 - \text{Accuracy})$ OR $(\text{FP} + \text{FN}) / \text{total}$
- ☐ **True Positive Rate ("Sensitivity" or "Recall"):** $\text{TP} / \text{actual yes}$
- ☐ **False Positive Rate:** $\text{FP} / \text{actual no}$
- ☐ **True Negative Rate ("Specificity" OR $(1 - \text{False Positive Rate})$):** $\text{TN} / \text{actual no}$
- ☐ **Precision:** $\text{TP} / \text{predicted yes}$
- ☐ **Prevalence:** $\text{actual yes} / \text{total}$



- ☐ **Accuracy:** $(\text{TP} + \text{TN}) / \text{total} = (100 + 50) / 165 = 0.91$ (Overall, how often is the classifier correct?)
- ☐ **Error Rate OR Misclassification Rate:** $(1 - \text{Accuracy})$ OR $(\text{FP} + \text{FN}) / \text{total} = (10 + 5) / 165 = 0.09$ (Overall, how often is it wrong?)
- ☐ **True Positive Rate ("Sensitivity" or "Recall"):** $\text{TP} / \text{actual yes} = 100 / 105 = 0.95$ (When it's actually yes, how often does it predict yes?)
- ☐ **False Positive Rate:** $\text{FP} / \text{actual no} = 10 / 60 = 0.17$ (When it's actually no, how often does it predict yes?)
- ☐ **True Negative Rate ("Specificity" OR $(1 - \text{False Positive Rate})$):** $\text{TN} / \text{actual no} = 50 / 60 = 0.83$ (When it's actually no, how often does it predict no?)
 - ☐ **Precision:** $\text{TP} / \text{predicted yes} = 100 / 110 = 0.91$ (When it predicts yes, how often is it correct?)
- ☐ **Prevalence:** $\text{actual yes} / \text{total} = 105 / 165 = 0.64$ (How often does the yes condition actually occur in our sample?)
- ☐ **F-measure or balanced F-score (F1 score) is the harmonic mean of precision and recall. = 0.93**

How to remember?

Many cannot remember the difference between sensitivity, specificity, precision, accuracy, and recall, despite having encountered these phrases multiple times. These are very basic terms, but the names are unintuitive, thus many keep getting them mixed up. What's a decent approach to think about these ideas such that the names make sense?

For precision and recall, each is the true positive (TP) as the numerator divided by a different denominator.

Precision and Recall: focus on True Positives (TP).

- Precision: $\text{TP} / \text{Predicted positive}$

- **Recall:** TP / **R**eal positive

Sensitivity and Specificity: focus on Correct Predictions.
There is one concept viz., **SNIP SPIN**.

- **SNIP (SeNsitivity Is Positive):** $TP / (TP + FN)$

- **SPIN (SPecificity Is Negative):** $TN / (TN + FP)$

SNIP refers to Sensitivity.
SPIN refers to Specificity.

Receiver Operator Characteristic (ROC)



- ☐ An ROC curve (receiver operating characteristic curve) is a graph showing the performance of a classification model at all classification thresholds.

- ☐ This curve plots two parameters:

- 1) True Positive Rate
- 2) False Positive Rate

- ☐ An ROC curve plots TPR vs. FPR at different classification thresholds.

- ☐ Lowering the classification threshold classifies more items as positive, thus increasing both False Positives and True Positives.

AUC: Area Under the ROC Curve

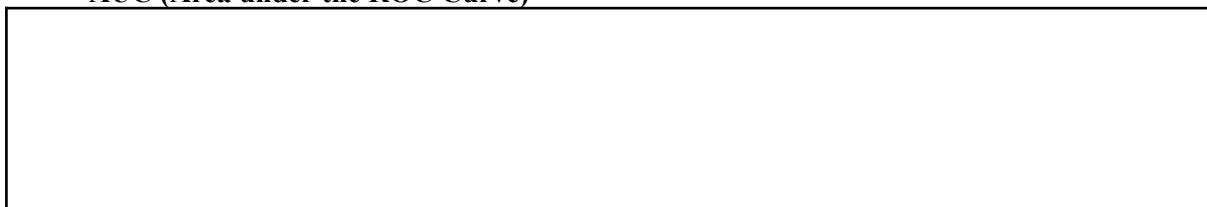


- ☐ An ROC curve plots TPR vs. FPR at different classification thresholds.

- ☐ AUC provides an aggregate measure of performance across all possible classification thresholds.

- ☐ AUC is the probability that the model ranks a random positive example more highly than a random negative example.

AUC (Area under the ROC Curve)



Regression Performance Evaluation Metrics:



Problems

Given the following actual values: [50, 70, 90, 110] and predicted values: [48, 72, 88, 115], calculate the MAE, RMSE, and R-squared. Which metric suggests the model has a better fit?



